Latency-Aware Adaptive QKD-PQC Key Establishment for Electronic Health Record Sharing in 6G-Edge Healthcare Networks: A Simulation-Based Evaluation Under Varying QKD Availability

Ramana Sree K V* and Verona Ann Mariya*

***Abstract*—The advent of cryptographically relevant quantum computers threatens the confidentiality of Electronic Health Records (EHRs) transmitted today, since an adversary can record encrypted traffic now and decrypt it once such a computer exists ("harvest-now, decrypt-later"). Two independent lines of defense have emerged: Post-Quantum Cryptography (PQC), which is always available but rests on computational hardness assumptions, and Quantum Key Distribution (QKD), which offers information-theoretic key material but is a finite, rate-limited, and interruptible physical resource. Prior work has separately shown that hybrid QKD-PQC key establishment is field-viable for healthcare data and that availability-aware fallback between QKD, hybrid, and PQC-only modes is effective for other critical infrastructure (power-grid communications), but no verified study has evaluated an adaptive fallback mechanism against an EHR-specific workload in a 6G-edge healthcare topology, where routine and emergency-access transactions carry different latency tolerances. This paper closes that gap. We present an adaptive QKD-PQC key-establishment architecture for EHR sharing, implement it alongside four baselines (classical elliptic-curve, PQC-only, QKD-only, and static QKD-PQC hybrid) in a validated discrete-event simulation using real ML-KEM-768 and ML-DSA-65 operations (via liboqs), and evaluate all five under a 30-configuration pilot sweeping QKD availability (0%, 50%, 100%) and device count (10, 1,000). The central empirical result is a sharp behavioral divergence under QKD unavailability: the static hybrid baseline collapses to a 0.20% successful-transmission rate (100/50,000 transactions), indistinguishable from the unmitigated QKD-only baseline, while the adaptive mechanism sustains 99.68% (49,842/50,000) by falling back to a PQC-only path. This resilience is not free: the adaptive baseline's mean key-establishment latency under degraded availability rises roughly two orders of magnitude, from 0.43 ms at full availability to 40–48 ms, driven almost entirely by the controller's bounded wait for QKD pool replenishment rather than by cryptographic cost, and it carries a fixed 8,890-byte key-establishment overhead regardless of which mode it ultimately selects. We report these results as a simulation study with real cryptographic execution on a general-purpose host; we make no claim about embedded or IoMT device performance, and we state explicitly where the current pilot's own instrumentation was found and fixed to be measuring the wrong system before it could be trusted. The pilot data, the fix that made it trustworthy, and the resulting trade-off surface are, to our knowledge, the first evidence-based characterization of this specific adaptive mechanism's cost under an EHR-shaped workload.**

***Index Terms*—Post-quantum cryptography, quantum key distribution, hybrid key establishment, 6G security, electronic health records, edge computing, adaptive key management, discrete-event simulation.**

Manuscript prepared from a validated simulation implementation and a 30-configuration, 5-repetition pilot experiment (757,500 simulated transactions). See the project repository for the complete implementation, raw results, and reproducibility manifest.

I. Introduction

Electronic Health Records (EHRs) carry a confidentiality requirement that most data does not: the requirement to remain confidential not just today, but for the years or decades a patient's clinical history remains sensitive. This long retention horizon is precisely what makes EHR traffic a target for the "harvest-now, decrypt-later" (HNDL) threat: an adversary who cannot break today's encryption can still record the ciphertext now and decrypt it later, once a cryptographically relevant quantum computer (CRQC) capable of running Shor's algorithm becomes available [1]. Unlike a threat that only matters once the capability exists, HNDL is a threat against data in transit *today*.

Two independent technical responses exist. Post-Quantum Cryptography (PQC) replaces number-theoretic hardness assumptions (factoring, discrete log) with problems believed hard even for a quantum computer, most prominently lattice problems underlying the NIST-standardized ML-KEM key-encapsulation mechanism and ML-DSA signature scheme [1], [2]. PQC is software-only, deployable without new hardware, and does not depend on physical channel conditions. Quantum Key Distribution (QKD) takes a different approach: rather than relying on computational hardness, it uses quantum-mechanical properties of photon transmission to produce shared secret key material whose security rests on the laws of physics rather than an assumption about adversary compute power. QKD is real and field-deployed — a 300 km trusted-node network has been reported in operation [3] — but it is also a bounded, rate-limited physical resource: fiber loss limits distance, key-generation rate is finite, and the channel itself can be degraded or interrupted by conditions outside any single party's control.

These two technologies motivate opposing institutional positions, and this paper's architecture is best understood as a

direct response to both. The U.S. National Security Agency has publicly argued against relying on QKD for national-security systems, citing three specific concerns: QKD lacks built-in entity authentication (it establishes a shared secret between two endpoints but does not, by itself, prove which endpoints they are); it requires expensive, dedicated physical infrastructure not needed by a software-only PQC deployment; and it is vulnerable to denial-of-service through deliberate channel perturbation — an adversary does not need to break QKD's underlying physics to defeat it operationally, only to disrupt the physical channel enough to prevent key generation. European standards bodies, through ETSI and the EuroQCI initiative, have taken the opposite institutional position, investing heavily in QKD specifically for the information-theoretic security guarantee no computational assumption can offer. Both positions identify real, substantiated concerns rather than talking past each other, and a system that treats QKD unavailability as an actively handled condition rather than a catastrophic failure directly addresses the NSA's stated denial-of-service objection — this is precisely what an availability-aware adaptive fallback mechanism, evaluated empirically in this paper, provides. Separately, authenticating QKD's classical control channel with a PQC signature scheme rather than assuming QKD's own physical security extends to it directly addresses the entity-authentication objection. Neither of these architectural choices resolves the NSA-versus-ETSI disagreement in the abstract; each is a concrete engineering response to one of the two specific, named objections raised by one side of it. This is the design space this paper investigates empirically, and Section IV states the corresponding threats (T3, T5, T6) formally.

A. Motivating Scenario

Consider a hospital's edge gateway aggregating EHR traffic from a mix of routine record access (a clinician reviewing a patient's history between appointments) and time-critical access (an emergency department pulling a patient's medication history during active treatment), over a QKD link shared with the hospital's classical network infrastructure. A static hybrid design — one that always requires both a QKD key contribution and a PQC key contribution before it will establish a session — offers a strong security property when the QKD link is healthy, but creates a single point of failure: if the QKD link degrades (fiber disruption, key-rate exhaustion under concurrent load, or a deliberately induced denial-of-service against the physical channel), every transaction requiring a new session key fails, including the emergency-department request, for as long as the QKD link remains degraded. A purely PQC-only design avoids this failure mode entirely but forgoes QKD's information-theoretic guarantee even when the QKD link is perfectly healthy and available at no cost. An adaptive mechanism that actively monitors QKD availability and falls back to PQC-only exactly when needed — and only when needed — is the design this scenario motivates, and it is the design this paper evaluates. The empirical question this paper asks is not whether such a mechanism is conceptually sound (Section III shows the general pattern is already established for other infrastructure), but what it actually costs, in latency and communication overhead, to buy that resilience under an EHR-shaped workload specifically — and whether a real implementation of it can be trusted to behave as designed, which Section XII shows is not a given.

Combining QKD and PQC is not new by itself. What has not been evaluated, as far as our literature search (Section III) was able to determine, is an *availability-aware fallback* mechanism — one that actively monitors QKD channel state and switches between a QKD-PQC hybrid mode and a PQC-only mode — applied to an EHR-specific workload in a 6G-edge healthcare topology. A closely related mechanism has been evaluated for power-grid communications [4], and hybrid QKD-PQC deployments for healthcare data exist in the literature [5]–[7], but the specific combination — adaptive fallback, tested against EHR transaction semantics that distinguish routine from emergency access, at 6G-edge scale — is, to our knowledge, unaddressed.

B. Contributions

This paper makes three contributions, each classified honestly by its type rather than claimed uniformly as novel:

- **C1 (architectural/integration).** An adaptive QKD-PQC key-establishment architecture for EHR sharing in a simulated 6G-edge topology, switching between a QKD-PQC hybrid mode and a PQC-only mode based on simulated QKD channel availability, with a bounded-wait policy that treats emergency-flagged transactions differently from routine ones. This integrates previously separate ideas — QKD+PQC hybrid combination, availability-aware fallback, and EHR-specific transaction criticality — rather than introducing a new cryptographic primitive.
- **C2 (experimental/extension).** A validated discrete-event simulation, using real ML-KEM-768 and ML-DSA-65 execution (not mocked), evaluating five baselines — classical, PQC-only, QKD-only, static hybrid, and adaptive hybrid — across a 30-configuration pilot sweeping QKD availability and device count, yielding 757,500 simulated transactions.
- **C3 (analytical).** An explicit characterization of the latency and overhead cost of adaptive switching under an EHR workload, including a documented case where the pilot's own instrumentation was found to be silently measuring the wrong behavior — and the fix, verification, and corrected results that followed. We report this methodological finding alongside the substantive one because a results section is only as trustworthy as the instrumentation that produced it.

C. Scope

This is a simulation study. Cryptographic operations are real — ML-KEM-768 and ML-DSA-65 execute through the reference `liboqs` library, AES-256-GCM and HKDF through a standard cryptography library — but they execute on the discrete-event simulation host, not on any embedded or IoMT hardware. We make no claim about the performance of these primitives on constrained microcontrollers, and Section XIII

states this boundary explicitly, together with every other simplification the network, QKD, and workload models make. The pilot reported here is a calibration study, not a completed systematic evaluation across the full parameter space; Section X explains why, and Section XV lists what a full study would add.

D. Paper Organization

Section III surveys related work and states the research gap. Section IV defines the threat model. Section V describes the system architecture and data flows. Section VI specifies the adaptive key-establishment mechanism. Section VII specifies the cryptographic design, including the hybrid key-derivation construction and its explicit, bounded security claim. Section VIII defines the five evaluated baselines. Section IX describes the simulation implementation. Section X describes the pilot experimental design. Section XI reports results. Section XII evaluates the paper's hypotheses against those results and discusses the instrumentation fix that made them trustworthy. Section XIII discusses threats to validity and scope limitations. Section XIV gives an informal security discussion. Section XV describes future work, and Section XVI concludes.

II. Background

This section gives the technical background needed to follow the rest of the paper without assuming prior familiarity with quantum key distribution, post-quantum cryptography, or 6G-edge network architecture. Readers already familiar with these areas may proceed to Section III.

A. The Harvest-Now, Decrypt-Later Threat

Public-key cryptography deployed today — RSA, elliptic-curve Diffie-Hellman, and their relatives — derives its security from computational problems (integer factorization, the discrete logarithm problem) that are believed intractable for classical computers but that a sufficiently large, fault-tolerant quantum computer running Shor's algorithm could solve in polynomial time. No such machine exists today at the scale required to threaten deployed key sizes. The threat this motivates is not, however, contingent on such a machine existing now. An adversary with the resources to store large volumes of encrypted traffic can record ciphertext today, retain it, and decrypt it retroactively once a cryptographically relevant quantum computer (CRQC) becomes available — a strategy commonly termed "harvest-now, decrypt-later" (HNDL). For data with a short useful lifetime, HNDL is not a practical concern: by the time a CRQC exists, the data is worthless. Electronic health records are the opposite case. A patient's clinical history routinely remains sensitive for decades, well within a plausible window for CRQC development under most technical forecasts. This is the reason EHR confidentiality is treated in this paper as a present-tense problem rather than a future one, and it is the primary motivation for including a post-quantum component in every cryptographic mode this paper evaluates except the deliberately non-quantum-resistant classical baseline (B1, Section VIII).

B. Post-Quantum Cryptography

Post-quantum cryptography (PQC) replaces the hardness assumptions above with problems believed hard even for a quantum adversary. The NIST post-quantum cryptography standardization process selected ML-KEM (based on the Module Learning-With-Errors problem over a structured lattice) as its primary key-encapsulation mechanism, standardized as FIPS 203 [1], and ML-DSA (based on the related Module-LWE and Module-SIS problems) as its primary signature scheme, standardized as FIPS 204 [2]. Both are software-only: they require no new physical hardware, no dedicated transmission medium, and no assumption about the physical layer beyond what any classical network already provides. Their principal costs relative to classical elliptic-curve cryptography are larger public keys, ciphertexts, and signatures (Table X reports the exact measured sizes this paper's simulation uses) and a comparatively young set of hardness assumptions, still under active cryptanalytic scrutiny relative to number-theoretic assumptions with decades of study.

C. Quantum Key Distribution

Quantum key distribution takes a fundamentally different approach: rather than relying on a computational hardness assumption at all, QKD protocols use the quantum-mechanical properties of individual photons — most fundamentally, the fact that an unknown quantum state cannot be measured without disturbing it (the no-cloning theorem) — to allow two parties to establish a shared secret key such that any eavesdropping attempt is, in principle, detectable. This gives QKD-derived key material an information-theoretic security property that does not depend on an adversary's computational power, present or future. That guarantee comes with three practical costs directly relevant to this paper's design. First, QKD requires dedicated physical infrastructure — typically fiber-optic links, though free-space and satellite variants exist — that a purely software-based PQC deployment does not. Second, the achievable key-generation rate falls off with distance due to photon loss in the transmission medium, which bounds both the rate and the practical range of a QKD link; a real, field-deployed trusted-node network operating over 300 km has been reported [3], illustrating that long-haul QKD is achievable but not that it is achievable at arbitrary distance or rate. Third, and most directly relevant to this paper's central mechanism, QKD's classical control channel — which carries the sifting, error-correction, and privacy-amplification traffic the protocol needs to distill a final secret key from raw quantum measurements — is not itself protected by the quantum channel's physical security, and must be independently authenticated or the entire protocol's security guarantee can be defeated by a classical man-in-the-middle attack on that control channel [8]. These three costs are the practical basis for the U.S. National Security Agency's publicly stated skepticism toward QKD for national-security use: the requirement for dedicated infrastructure, susceptibility to denial-of-service through physical channel disruption, and (absent independent authentication) the lack of built-in entity authentication. A system that treats QKD unavailability as an

TABLE I
GLOSSARY OF ACRONYMS.

Acronym	Meaning
AEAD	Authenticated Encryption with Associated Data
CRQC	Cryptographically Relevant Quantum Computer
EHR	Electronic Health Record
ETSI	European Telecommunications Standards Institute
HKDF	HMAC-based Extract-and-Expand Key Derivation Function
HNDL	Harvest-Now, Decrypt-Later
IoMT	Internet of Medical Things
KEM	Key-Encapsulation Mechanism
KDF	Key-Derivation Function
MITM	Man-in-the-Middle
ML-DSA	Module-Lattice-Based Digital Signature Algorithm
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism
NIST	National Institute of Standards and Technology
PQC	Post-Quantum Cryptography
QKD	Quantum Key Distribution

TABLE II
NOTATION.

Symbol	Meaning
L	Current QKD key-pool level, in bits
C	QKD key-pool capacity, in bits
r	QKD key-generation rate, in bits/second
Δt	Elapsed simulated time between pool updates
a	Normalized QKD pool availability, $L/C \in [0, 1]$
τ_{hybrid}	Availability threshold above which hybrid mode is selected immediately
τ_{wait}	Availability threshold below which the pool is treated as exhausted (no wait attempted)
T_{wait}	Maximum bounded-wait duration for pool replenishment
K_{QKD}, K_{PQC}	QKD-derived and ML-KEM-derived secret key material
C_T	HKDF context label (session/transaction identifier + negotiated mode)
a	Configured nominal QKD availability level for a pilot cell (0.0, 0.5, or 1.0)

expected, actively handled condition rather than an exceptional failure, and that authenticates QKD's classical channel independently of the quantum channel itself, directly addresses the second and third of these objections; this is the design this paper evaluates.

D. 6G-Edge Computing for Healthcare

Sixth-generation (6G) mobile networks are, as of this writing, not a finalized standard; the term in this paper refers to the anticipated combination of very high bandwidth, low access latency, and pervasive edge-compute placement that 6G research generally targets, particularly for latency-sensitive applications. Edge computing — placing compute and storage resources physically close to where data originates, rather than routing every transaction to a centralized cloud or data-center facility — is a natural fit for healthcare IoMT (Internet of Medical Things) deployments, where a hospital's own edge infrastructure can aggregate telemetry from bedside and wearable devices before it reaches a hospital's core EHR systems. This paper's architecture (Section V) places PQC and adaptive key-establishment logic at exactly this edge tier — the Healthcare Edge Gateway — rather than at the IoMT endpoint itself, on the premise that endpoint devices are resource-constrained in ways an edge gateway is not. We state explicitly in Section XIII that this premise is a design decision inherited from the broader system model, not a claim this paper's simulation independently tests, since the simulation contains no device-class timing model of its own.

E. Glossary

Table I lists acronyms used throughout this paper, for reference.

F. Notation

Table II summarizes the notation used throughout the remainder of this paper.

III. Related Work and Research Gap

A. Literature Verification Note

Before surveying the literature, we state plainly how it was gathered, because the strength of a related-work section depends on how its claims were verified. This survey rests on approximately 25 targeted live searches conducted across the project's research phase, not the exhaustive 30–50 paper systematic review that a full literature review would typically require. Three sources ([5], [9], [10]) were independently confirmed via resolved DOI and, in one case, full-text retrieval. A further set of sources — [3], [4], [6]–[8], [11]–[15] — were located via live web search with real, specific titles, authors (where available), and venues identified, but were not independently re-verified against full text at publication-grade confidence. We cite them at that confidence level and say so here rather than silently upgrading it. Where a claim in this section rests specifically on one of these partially-verified sources, we say so in the text. No citation in this paper was invented; where an author list or exact identifier could not be confirmed, it is simply omitted from the bibliography entry rather than guessed.

B. Post-Quantum and Hybrid QKD-PQC Security for Healthcare

Post-quantum cryptography for healthcare data is an active and increasingly field-validated area. Devaraj et al. propose a multi-layered PQC architecture (ML-KEM and ML-DSA-class primitives) for 6G-enabled smart hospitals [5]; this is the most directly comparable prior architecture we identified, though it is PQC-only and does not incorporate a QKD component. A broader review of post-quantum cryptography for healthcare data, connected devices, and digital health infrastructure appears in [9], and a systematic review of post-quantum authentication specifically for the Internet of Medical Things (IoMT), screening 63 studies, appears in [10].

Hybrid QKD-PQC combination for healthcare specifically has field-level precedent. Roosan et al. describe a

DAG-blockchain architecture integrating PQC and QKD with privacy-preserving mechanisms for telehealth patient records [6]. Papadopoulos et al. report a practical, field-piloted deployment of hybrid QKD and PQC for off-site hospital data over a real QKD link (reported as built on the HellasQCI/EuroQCI infrastructure) [7] — a real deployment, not a simulation, though the fallback behavior of that deployment under QKD channel degradation is not established by the source material available to us. A further field-deployed report describes secure medical data transmission over a 140 km real-world fiber network combining entanglement-based QKD with end-to-end PQC authentication [14]. None of these three establishes an *availability-aware fallback* mechanism between hybrid and PQC-only modes; where fallback behavior is discussed at all, it is not the paper's evaluated contribution.

Beyond healthcare, Mahesh and Mishra propose a patient-centric EHR system using multiple combined lattice-based PQC algorithms on a blockchain substrate [12]; this is PQC-only, with no QKD component. Maqsood et al. evaluate PQC schemes (including a lattice-based KEM and a hash-based signature scheme) specifically for resource-constrained wearable IoMT devices, with real hardware benchmarking [13] — a hardware feasibility study, distinct in kind from the simulation-based evaluation in this paper, and precisely the kind of evidence Section XIII notes this paper does *not* provide.

C. QKD Classical-Channel Authentication and QKD Deployment

A design decision in this paper's architecture (Section V) is that the QKD subsystem's classical control channel is authenticated using ML-DSA rather than classical public-key infrastructure. This choice is grounded in Atutxa, Sanz, and colleagues' finding that a QKD deployment's classical channel requires authentication independent of the QKD protocol's own physical-layer security, in the context of a multi-site 5G/6G quantum-safe network [8]. We treat this as a design assumption inherited from that source, not as an independently re-derived result.

On QKD's physical deployment characteristics, Clason et al. report a deployed trusted-node QKD network operating over 300 km with a multi-site topology [3]. We use this source only as loose plausibility context for our QKD pool's simulated parameters (Section IX), not as a literal topology or as literature-derived values for the pool's capacity or generation rate, which remain explicitly labeled as modeled assumptions throughout this paper. Kudaloor and Aijaz report an experimental evaluation of post-quantum cryptography intended for 6G contexts [15], consistent with this paper's use of ML-KEM/ML-DSA as the PQC layer, though their evaluation is not EHR-specific.

D. Adaptive Fallback in Critical Infrastructure

The single closest prior evaluation methodology we identified is Zhu's techno-economic feasibility analysis of QKD for power-system communications [4], which evaluates QKD-only, PQC-only, and hybrid modes against power-grid traffic (GOOSE/PMU-class messages) using stochastic modeling, with fallback fraction, latency, and overhead as core reported results. This is, to our knowledge, the most rigorous existing instance of the general evaluation methodology this paper adopts — but for power-grid traffic, not EHR data, and without an EHR-specific transaction-criticality axis (routine versus emergency access) of the kind Section IX-F introduces. Spooren et al. describe a layered WireGuard-and-Rosenpass construction operating over an ETSI QKD API with an explicit fail-safe design for a generic multi-hop network, evaluated in a lab testbed and simulation [11]; this establishes fail-safe layered QKD-PQC design as a real, evaluated pattern, though not for a healthcare workload or a 6G-edge topology.

TABLE III
BROADER LITERATURE LANDSCAPE (20 SCREENED CANDIDATE SOURCES).

Dimension addressed	Count (of 20)
QKD	8
PQC	12
Healthcare application	10
6G / 5G-and-beyond networking	7
QKD *and* PQC together	2
Healthcare *and* 6G together	2
All four dimensions	1 (venue unconfirmed)

E. Broader Literature Landscape

Beyond the sources discussed individually above, this project's search process screened a broader set of 20 candidate sources across the QKD, PQC, healthcare, and 6G dimensions, summarized in Table III. Of these, 8 address QKD, 12 address PQC, 10 address a healthcare application, and 7 address 6G or 5G-and-beyond networking; only 2 address both QKD and PQC together, and only 2 address both a healthcare application and 6G/5G networking together. Exactly one candidate source flagged all four dimensions simultaneously — and it is specifically the one source in this set whose publication venue could not be independently confirmed as a recognized, indexed venue, which is itself worth reporting rather than silently omitting: the nearest apparent match to this paper's full combination of dimensions is the one match we have the least confidence actually exists as a credible, citable publication. We treat this as further, if inconclusive, support for the research gap in Table V, not as proof that no closer match exists anywhere in the broader literature.

F. Community Interest

Independent of the specific papers surveyed above, the research community has identified this general area as a live target: at the time of this project's literature search, IEEE Journal of Biomedical and Health Informatics had an open special-issue call, "Quantum Key Distribution for Secure Healthcare Information in 6G-Enabled Systems," explicitly soliciting work on QKD-in-6G approaches covering EHR data, IoMT, and hybrid classical-quantum encryption. We report this as evidence that the field has identified the general problem

area as significant, not as evidence that the specific gap in Table V is unaddressed — a call for papers is not itself a result, and competing work already in submission at the time of our search would not necessarily be visible to it.

G. Detailed Comparison

Table IV extends the positioning in Table V with additional dimensions — whether a source addresses key-management lifecycle, endpoint/channel authentication, QKD failure handling specifically (as opposed to hybrid combination generally), and whether latency and overhead are evaluated quantitatively — for the sources most central to this paper's positioning.

H. Research Gap

Table V summarizes where the closest related work sits relative to the combination this paper evaluates. No source we identified combines an availability-aware QKD-PQC fallback mechanism, an EHR-specific workload with distinct routine and emergency-access latency tolerances, and a 6G-edge network topology, evaluated with a working simulation across concurrent-device scale. Zhu [4] provides the fallback-mechanism evaluation methodology but for a different traffic class; Devaraj et al. [5] and the healthcare-QKD deployments [6], [7], [14] provide the healthcare-application and field-viability grounding but not the adaptive-fallback evaluation. This is the gap the present paper addresses.

IV. Threat Model

A. Adversary Model

We state the adversary's assumed capabilities explicitly, since several of this paper's design decisions (Section V) only make sense relative to a specific capability boundary. The adversary is assumed to have: full passive access to the classical network path between any two architectural components (Section V's trust boundaries), including the ability to record ciphertext indefinitely for later analysis (the premise of T1); the ability to actively intercept, inject, modify, or replay messages on the classical control channel used for QKD sifting and error correction, and on the mode-synchronization channel (T3); the ability to physically or environmentally interfere with a QKD link's photon transmission, degrading or eliminating its key-generation rate, without necessarily being detected as doing so through means outside this paper's scope (T5, T6); and control of a legitimately provisioned IoMT endpoint whose credential remains valid despite the endpoint itself being compromised (T4). The adversary is explicitly *not* assumed to possess a cryptographically relevant quantum computer at the time of the attack — T1's harm model is that such a capability may exist *later*, against ciphertext captured now, which is precisely what distinguishes HNDL from a conventional confidentiality threat and is why PQC, not merely strong classical cryptography, is the relevant mitigation. The adversary is also not assumed to have compromised any endpoint's private key material through means outside this threat model (side-channel extraction, supply-chain compromise of the cryptographic library itself, or physical device theft), nor to have broken the PQC hardness assumption underlying ML-KEM or ML-DSA — both are treated as out-of-scope external risks (Section IV), not threats this architecture claims to defend against. This capability boundary is what makes the specific threats T1–T6 below a coherent, bounded set rather than an arbitrary list: each maps to one concrete capability stated in this paragraph, and no defense claimed anywhere in this paper (Section XIV) is asserted against a capability outside it.

B. In Scope

The architecture and evaluation in this paper address the following threats:

- **T1 — Harvest-now, decrypt-later.** A quantum-capable adversary passively records classical-channel traffic today with the intent to decrypt it once a CRQC becomes available. This is the primary motivation for including PQC in every mode.
- **T2 — Passive interception.** Passive eavesdropping on both the classical application channel and, where present, the quantum channel.
- **T3 — Classical-channel man-in-the-middle.** An adversary that can intercept and modify the QKD classical control channel (sifting, error-correction, and privacy-amplification messages) can defeat QKD's security guarantee regardless of the quantum channel's own physical security, if that classical channel is not itself authenticated. This motivates the design decision (Sections V, III) to authenticate the classical control channel with ML-DSA.
- **T4 — Compromised or malicious IoMT endpoint.** An endpoint device that has a valid credential but is compromised, and attempts to inject falsified telemetry or interfere with the adaptive fallback logic.
- **T5 — QKD channel unavailability.** Fiber disruption, key-rate exhaustion, or adverse physical conditions that degrade or eliminate QKD key material availability. This is modeled as a threat/fault condition to be actively handled, not merely as an operational inconvenience — it is the condition this paper's central mechanism exists to address.
- **T6 — Downgrade/fallback attack.** An adversary that deliberately induces or spoofs QKD unavailability specifically to force the system into a weaker mode. Because the adaptive mechanism's fallback to PQC-only is a legitimate, expected behavior (not itself a vulnerability, since PQC-only remains quantum-resistant), this threat's practical severity depends on whether the forced mode is itself adequately secure — which, for a system whose fallback target is PQC rather than an unauthenticated or classical-only mode, it is, under the PQC hardness assumption.

C. Explicitly Not Defended Against

Consistent with the scope boundary in Section XIII, this paper's threat model and evaluation explicitly exclude:

TABLE IV
DETAILED COMPARISON ACROSS EVALUATION DIMENSIONS.

Source	Key mgmt.	Auth.	QKD failure handling	Latency eval.	Overhead eval.
Roosan et al. [6]	Blockchain-ledger	ABE + contracts	Not addressed	Not primary metric	✓
Papadopoulos et al. [7]	Unclear (field pilot)	Unclear	Unverified	Unverified	Unverified
Spooren et al. [11]	✓(explicit)	✓	✓(fail-safe design)	✓	✓
Atutxa & Sanz et al. [8]	Not central	✓(core contribution)	Not addressed	Not central	Not central
Zhu [4]	SLA/buffer-driven	Not focus	✓(stochastic model, core result)	✓(core result)	Implicit
This work	✓(Section IX)	✓(ML-DSA)	✓(Algorithm 1)	✓(Section XI)	✓(Section XI)

TABLE V
POSITIONING AGAINST THE CLOSEST RELATED WORK. ✓ = EXPLICIT/EVALUATED, ✗ = ABSENT, — = NOT APPLICABLE/NOT ESTABLISHED FROM THE AVAILABLE SOURCE.

Source	Hybrid QKD-PQC	Adaptive fallback	EHR + 6G-edge
Devaraj et al. [5]	PQC-only	—	✓(6G)
Roosan et al. [6]	✓	—	✓(partial)
Papadopoulos et al. [7]	✓(field)	—	✓(partial)
Spooren et al. [11]	✓	✓(fail-safe)	—
Zhu [4]	✓	✓(evaluated)	— (power grid)
This work	✓	✓(evaluated)	✓

TABLE VI
SYSTEM COMPONENTS.

Component	Role
EHR Client / Application	Initiates EHR transactions (read/write/share) on behalf of a clinician or patient-facing application.
IoMT / Healthcare Endpoint	Contributes telemetry (e.g., vitals) into the EHR pathway. Always PQC-only by design — no QKD hardware is assumed at the endpoint.
6G Access / Network Layer	A simulated wireless access abstraction (latency, throughput, loss); not a real 6G protocol stack.
Healthcare Edge Gateway	Aggregates client/device traffic; hosts the Adaptive Security Layer, the PQC subsystem, and the key-management component.
Adaptive Security Layer	Selects hybrid vs. PQC-only mode (Section VI); present at both the Edge Gateway and Hospital Server, which must agree via a mode-sync handshake.
QKD Subsystem / Pool	Simulated quantum-channel abstraction producing symmetric key material into a bounded pool at a configurable rate.
PQC Subsystem	ML-KEM key encapsulation and ML-DSA signatures; software-only, always available.
Hospital / EHR Server	Holds synthetic EHR data; terminates the secure session; authenticates and decrypts.
Key-Management Component	Orchestrates key lifecycle — generation, pooling, rotation, expiration, destruction — for both QKD- and PQC-derived material.

- Physical-layer QKD attacks (e.g., detector blinding or other side-channel attacks against real QKD hardware) — outside what a simulation study can meaningfully model, since no physical QKD hardware exists in this project.
- Compromise of the underlying PQC hardness assumption itself — treated as an accepted external risk, consistent with the broader PQC research community's position.
- Insider threats using legitimately issued credentials.
- Classical network- or operating-system-level compromise outside the modeled key-establishment and transport layers.
- Physical QKD hardware development, complete 6G standardization, clinical outcomes, real hospital deployment, or formal regulatory/compliance certification (e.g., HIPAA, GDPR) — these motivate the problem but are not evaluated or claimed.

V. SYSTEM ARCHITECTURE

A. Components

The architecture models nine logical components, summarized in Table VI, connected as shown in Table VII. A tenth element, the *authenticated classical control channel*, carries QKD sifting, error-correction, and privacy-amplification traffic together with the PQC handshake; it is treated as an interface rather than a standalone stateful component.

B. Component Placement Rationale

Two placement decisions in Table VI are not arbitrary and are worth justifying explicitly, since an equally plausible alternative placement exists for each and a reader auditing this architecture should be able to see why the alternative was not chosen. First, the Adaptive Security Layer is instantiated at *both* the Healthcare Edge Gateway and the Hospital/EHR Server, rather than only at the gateway with the server simply trusting whatever mode the gateway reports. A gateway-only placement would be simpler — one fewer component instance, no mode-sync handshake required — but it would also mean the server derives or accepts a session key without independently confirming which mode produced it, which reopens exactly the kind of trust-without-verification gap Section V's

TABLE VII
CONNECTIONS BETWEEN COMPONENTS. "ADAPTIVE DECISION" IDENTIFIES WHERE THE HYBRID-VS-PQC-ONLY CHOICE IS MADE.

Connection	Data	key-establishment	Encryption	Authentication	Adaptive decision
IoMT ↔ Edge Gateway	Telemetry	At gateway (ML-KEM)	At device (AEAD)	Device cert (ML-DSA) → gateway	Not made here — always PQC-only
EHR Client ↔ Edge Gateway	EHR request/response	At gateway	At sender (AEAD)	Mutual (ML-DSA)	**Made here**
Edge Gateway ↔ 6G Access	Already-encrypted payload	—	—	—	—
Edge Gateway ↔ Hospital Server	Encrypted payload + mode-sync	Jointly (QKD draw + ML-KEM)	At sender	Mutual (ML-DSA)	**Confirmed/mirrored here**
QKD Subsystem ↔ Classical Channel	Sifting/error-correction/privacy-amplification	—	—	ML-DSA	—

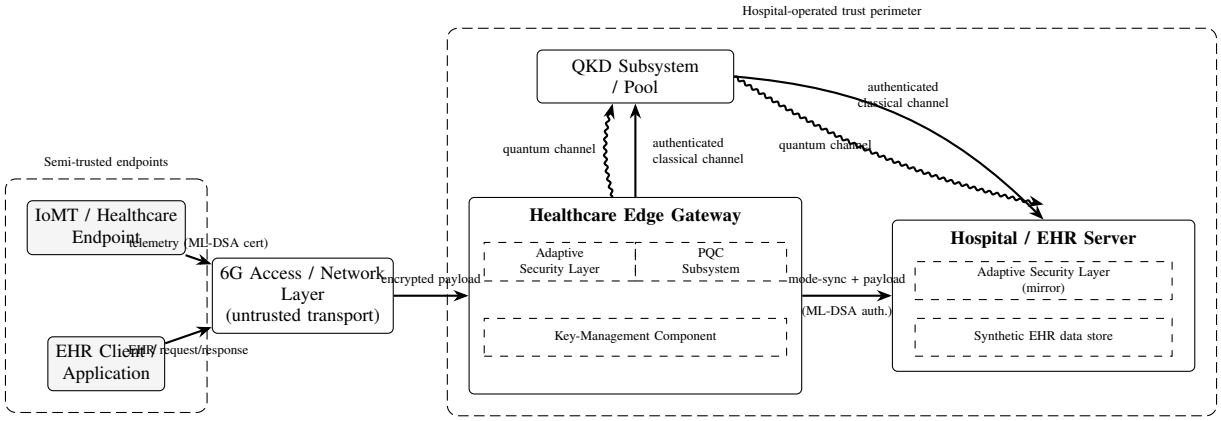

Fig. 1. Overall system architecture. Solid arrows are authenticated classical channels; wavy arrows denote the (simulated) quantum channel. The 6G access/network layer is treated as untrusted transport; confidentiality depends on end-to-end application-layer encryption, not on any property of the network. This diagram is a direct visualization of Table VI (components) and Table VII (connections) — it introduces no element not already specified in those tables.

trust-boundary discussion argues against at every other boundary in this architecture. Placing the decision logic at both ends and requiring an authenticated mode-sync handshake to confirm agreement (Section V's Data Flow) is the more conservative choice, paid for with one additional signature round trip that Section XI shows is a fixed, small share of B5's total overhead (3,309 of 8,890 bytes) relative to the correctness property it buys. Second, the Key-Management Component is placed at the Edge Gateway, not at the IoMT endpoint, which is consistent with — and partly motivates — the endpoint being PQC-only by design (Table VI) rather than a QKD-capable node in its own right. Centralizing key management at the gateway means a resource-constrained endpoint never needs to run pool-state bookkeeping, key rotation, or the adaptive decision logic itself; it only needs to hold and use a session key the gateway has already established on its behalf. This placement is a direct consequence of treating IoMT endpoints as computationally unconstrained-but-untrusted for cryptographic purposes only insofar as they can perform the PQC operations Section XIII discusses — it does not, by itself, resolve whether a real embedded IoMT device could feasibly run even that PQC-only path, which remains the open hardware-feasibility question this paper explicitly does not answer (Section XIII).

C. Trust Boundaries

Five trust boundaries structure the architecture, each with an explicit statement of what is trusted, what is not, and what protects the asset at stake. **IoMT endpoint to Edge Gateway**: the Edge Gateway is trusted (within the hospital's operated perimeter); the endpoint is treated as semi-trusted, since a compromised device retains a valid credential (Threat T4) — the gateway authenticates the credential but does not extend blanket trust to the device's reported data merely because the credential validates, and the attack surface this boundary must tolerate includes device firmware compromise and malicious data injection under an otherwise-valid credential. **Edge Gateway to 6G network**: neither side is fully trusted — the network itself is treated as untrusted transport by design, so confidentiality depends on end-to-end application-layer encryption (Section VII) rather than on any security property of the access network, which is also why the simulated 6G layer is modeled purely as a latency/throughput/loss abstraction (Section IX-E) rather than requiring its own cryptographic treatment; the residual attack surface is passive eavesdropping (motivating T1) and traffic analysis, explicitly out of scope. **6G network to hospital infrastructure**: the hospital infrastructure is trusted within its own perimeter; everything upstream, including the 6G radio access and core network, is not — the hospital's ingress

point re-validates authentication on every arriving request rather than extending implicit trust from having arrived via the hospital's own network path, protecting against ingress-point compromise and malformed or replayed traffic. **QKD subsystem to classical network**: the authenticated classical control channel is itself the trust anchor for QKD's entire security guarantee; an unauthenticated classical channel would let a man-in-the-middle defeat QKD regardless of the quantum channel's own physical security (Threat T3), which is why this channel is authenticated with ML-DSA rather than classical PKI, a design choice grounded in [8] rather than independently derived here. **Hospital EHR Server to clients**: the server is the root of trust for patient data; every client, including already-authenticated ones acting beyond their authorized scope, is untrusted by default — every request is authenticated per-session with no implicit trust extended from network position, loosely following zero-trust principles, though full access-control policy modeling is explicitly out of this paper's scope.

D. Data Flow

1) Normal flow (QKD available): A transaction (read, write, or share) is initiated; the Adaptive Security Layer checks whether an existing valid session key can be reused within its rotation window (Section IX) or whether new key-establishment is needed. If new, the layer queries current QKD pool state, and with availability above threshold selects the hybrid mode. The QKD subsystem supplies fresh key material from the pool while the PQC subsystem concurrently performs ML-KEM encapsulation and ML-DSA-based mutual authentication of both the classical QKD control channel and the application session; the session key is derived per Section VII. Edge Gateway and Hospital Server perform a mode-sync handshake to confirm agreement on the active mode. The EHR payload is encrypted at the sending endpoint under AEAD, transmitted through the simulated 6G-edge network, and authenticated and decrypted at the Hospital Server. Session state (key-usage counters, QKD pool debit, rotation clock, transaction mode, and latency) is logged, feeding both the adaptive controller's future decisions and the metrics collected for this evaluation.

2) Degraded flow (QKD available but below full capacity): Identical through pool state query. At the decision step, a graduated policy applies rather than a binary switch: for non-emergency transactions, the key-management component may wait briefly, bounded by a timeout, for pool replenishment before deciding (Section VI); emergency-flagged transactions never wait. Once a mode is settled, the remainder of the flow proceeds as in the normal case, and the transaction is logged as degraded-mode for the fallback-frequency metric.

3) QKD-unavailable flow (pool exhausted or below minimum threshold): The adaptive layer selects PQC-only mode immediately, without waiting. key-establishment proceeds using ML-KEM alone; the classical control channel remains ML-DSA-authenticated regardless of outcome, since it exists independent of which mode is ultimately chosen. The remainder of the flow is unchanged.

4) Failure handling: A QKD pool read failure or an unresponsive QKD subsystem is treated as zero availability, proceeding to PQC-only — a fail-*safe*, not fail-closed, design choice that prioritizes availability, and one that directly targets Threat T6 (an adversary inducing apparent QKD unavailability): because the fallback target is PQC, not an unauthenticated or unencrypted mode, forcing this fallback does not itself compromise confidentiality. A PQC key-establishment failure causes the transaction to fail after a bounded number of retries, counted against the successful-transmission-rate metric; it never silently falls back to a weaker-than-PQC mode. A mode mismatch between Edge Gateway and Hospital Server (e.g., from message loss during mode-sync) causes the transaction to fail closed — rejected outright rather than decrypted under a guessed mode.

VI. Adaptive Key-Establishment Mechanism

The adaptive mechanism is the paper's central design component. Algorithm 1 specifies its decision logic precisely.

A. Inputs

The controller's inputs are: the current QKD key-pool level, expressed as a fraction $a \in [0, 1]$ of target pool capacity; the recent QKD key-generation rate (reflecting channel quality); and the transaction's criticality class (routine or emergency), drawn from the EHR workload model (Section IX-F). Criticality is the input that differentiates this policy from a direct transplant of the power-grid mechanism in [4], which has no criticality axis; it is also the input most directly responsible for this paper's second hypothesis (Section XII) being testable at all. Network congestion is deliberately excluded as a separate input, since its effect is already captured indirectly through recent key-establishment latency history and adding it again would double-count the same signal. Device compute capability is likewise excluded as a per-transaction input, because in this system model every endpoint is PQC-only by construction — capability does not vary per transaction.

B. Worked Example

The pilot's actual configuration uses $\tau_{\text{hybrid}} = 0.3$, $\tau_{\text{wait}} = 0.1$, and $T_{\text{wait}} = 0.05\,\text{s}$ (Table XIII). Tracing Algorithm 1 through four concrete pool states makes the decision boundary explicit. At $a = 0.6$, the first condition is satisfied immediately and the controller returns HYBRID without consulting criticality or waiting — this is the common case at the pilot's higher configured availability levels and requires no further branching. At $a = 0.05$ for an emergency transaction, the first condition fails ($0.05 < 0.3$) and the second succeeds, so the controller returns PQCONLY immediately regardless of how close a is to τ_{wait} — emergency transactions never enter the wait branch, by construction, which is the mechanism responsible for hypothesis H2's predicted latency separation between criticality classes (Section XII). At $a = 0.18$ for a routine transaction, the first condition fails, the second does not apply (routine, not emergency), and the third succeeds ($0.18 \geq 0.1$): the controller waits up to 50 ms for the pool to replenish via

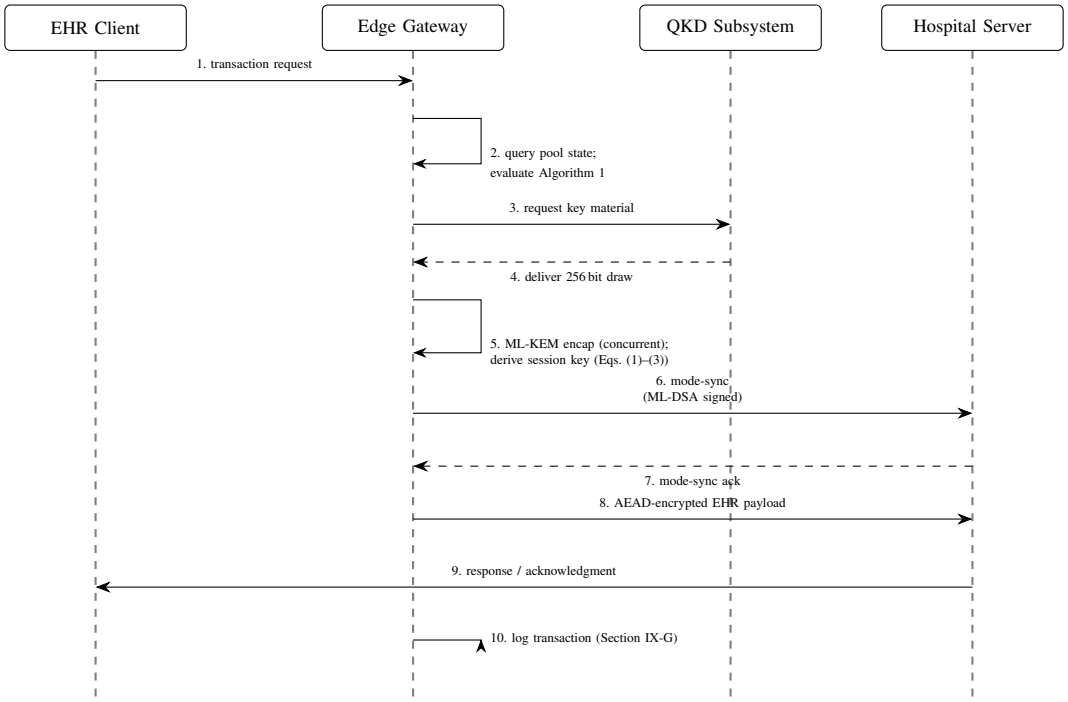

Fig. 2. Adaptive key-establishment message sequence for the normal flow (QKD available), corresponding directly to Section V's Data Flow description. Steps 3–4 (QKD draw) and step 5 (ML-KEM encapsulation) occur concurrently, not sequentially, as drawn here only for message-ordering clarity. In the degraded and unavailable flows (also described in this section), step 2's decision instead routes through the bounded-wait or immediate-PQC-only branches of Algorithm 1, and steps 3–4 are skipped entirely when the controller resolves to PQC-only.

Algorithm 1 Adaptive mode selection

Require: pool fraction a, criticality c, thresholds $(\tau_{\text{hybrid}}, \tau_{\text{wait}}, T_{\text{wait}})$

if $a \geq \tau_{\text{hybrid}}$ **then**
  return HYBRID
end if
if $c =$ EMERGENCY **then**
  return PQCONLY {never waits}
end if
if $a \geq \tau_{\text{wait}}$ **then**
  wait up to T_{wait} for pool replenishment
  if $a_{\text{after}} \geq \tau_{\text{hybrid}}$ **then**
    return HYBRID
  else
    return PQCONLY
  end if
end if
return PQCONLY {pool effectively exhausted}

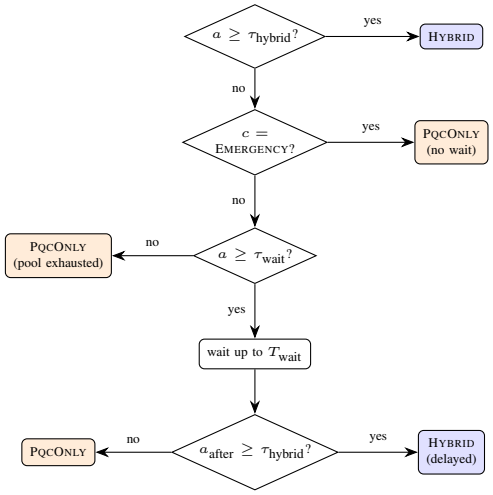

Fig. 3. Decision flowchart for Algorithm 1. Blue terminals resolve to HYBRID; orange terminals resolve to PQCONLY. Emergency-flagged transactions (the middle decision) are structurally prevented from ever reaching the wait branch, which is the mechanism underlying hypothesis H2 (Section XII). Section VI-B traces four concrete pool states through this exact diagram.

the simulated regeneration rate r (Section IX-D); if the pool crosses 0.3 within that window the controller returns HYBRID at the cost of the elapsed wait, and if it does not, PQCONLY. This is the specific path Section XII's instrumentation-fix narrative and the bounded-wait-path regression tests (Table XI) exist to verify, since it is also the path a bookkeeping bug most easily hides in — a wait that is computed but never actually charged to simulated time, as Section XII describes, produces a plausible-looking but silently wrong result specifically in this branch. Finally, at $a = 0.02$ for a routine transaction, all three conditions fail and the controller falls through to the unconditional final line, returning PQCONLY without waiting — the pool is judged effectively exhausted, and waiting would only delay an outcome the controller can already determine will not change.

C. Key Establishment Given the Selected Mode

In HYBRID mode, N bits are drawn from the QKD pool (debiting it) concurrently with an ML-KEM encapsulation, and the session key is derived per Section VII. In PQCONLY mode, only the ML-KEM shared secret contributes. The classical control channel is ML-DSA-authenticated in both cases.

D. Switching Granularity

Mode selection is made per session (a key-validity window bounded by a transaction count or elapsed time, whichever comes first) rather than per transaction or per fixed time window. Per-transaction switching would maximize responsiveness to QKD state changes but would make measured latency and overhead dominated by key-establishment handshake cost rather than by the mechanism's actual adaptive behavior — directly undermining what this evaluation measures. Per-time-window switching would introduce an additional free parameter with no literature-grounded value. Per-session switching reuses the key rotation policy the system already needs and lets the pool absorb QKD's naturally bursty generation without constant re-negotiation. This is a systems-engineering trade-off, not a claim requiring independent literature validation, and Section XV identifies it as a candidate for future sensitivity analysis.

E. Design Principles

Three recurring design choices, made independently at different points in the system and for different immediate reasons, share a common structure worth naming explicitly rather than leaving implicit across separate sections. **Structural fairness across baselines.** Every baseline shares the same AEAD payload encryption path, the same metrics-collection instrumentation, and the same simulated network layer; the five baselines differ only in how the session key is established (Section VIII). B4 in particular is deliberately prevented from ever falling back to PQC-only under QKD scarcity, even though doing so would improve its measured success rate, because B4 exists specifically to answer "what does a *non*-adaptive hybrid design cost under QKD scarcity" — letting it adapt would collapse the comparison this paper's central claim depends on (Section VIII). Fairness here means holding every variable but the one under test constant, not maximizing any individual baseline's own performance. **Fail-safe, not fail-closed, under QKD unavailability.** When the QKD pool cannot supply the adaptive controller's threshold within the wait budget, B5 falls back to PQC-only key establishment rather than refusing to establish a session at all (Section VI). This is a deliberate choice to preserve system availability over a stronger but more fragile guarantee, grounded in the healthcare-specific cost asymmetry between a delayed or degraded-security transaction and a clinician unable to reach a patient's record during an emergency (Section IV); it is not free, since PQC-only key establishment is not the paper's HNDL-optimal outcome, but it is the deliberate trade-off this paper's threat model motivates rather than an oversight. **Fail-closed on internal inconsistency.** The opposite choice applies wherever the failure indicates the system's own internal state is untrustworthy rather than merely degraded: the ML-KEM shared-secret consistency check (Section VII-E) aborts rather than deriving a key from mismatched secrets, and the mode-synchronization handshake (Section V) is authenticated specifically so that a disagreement between endpoints about which mode was selected is detectable rather than silently producing two sides that believe different things about their own shared key. The distinction between these last two principles is not arbitrary: QKD scarcity is an expected, anticipated operating condition the system is explicitly designed to tolerate, while a shared-secret mismatch or an unauthenticated mode-sync message indicates something the system's own design assumptions say should not be possible under correct operation — expected degradation is tolerated; violated invariants are not. Naming this distinction explicitly is intended to make the system's failure-handling behavior predictable to a reader auditing it from the outside, rather than requiring the same conclusion to be independently re-derived from scattered exception-handling code.

TABLE VIII
CRYPTOGRAPHIC PRIMITIVES.

Primitive	Role
ML-KEM-768	PQC key encapsulation (all baselines except B1, B3). NIST FIPS 203 [1]. Security under Module-LWE.
ML-DSA-65	PQC digital signatures — endpoint authentication, QKD classical-channel authentication, mode-sync. NIST FIPS 204 [2].
X25519 / Ed25519	Classical elliptic-curve key exchange and signatures (Baseline B1 only) — included specifically because it lacks post-quantum resistance, which is the point of that baseline.
AES-256-GCM	Authenticated payload encryption, identical across every baseline once a key is derived.
HKDF (SHA-256)	Session-key derivation from one or two input secrets [16].
QKD-generated key material	Contributes entropy to the hybrid session key when available; modeled as a bounded resource (Section IX-D), not simulated at the physical layer.

VII. CRYPTOGRAPHIC DESIGN

No new cryptographic primitive is proposed anywhere in this paper. Table VIII summarizes the primitives used and their role.

A. Algorithm Selection Rationale

ML-KEM-768 and ML-DSA-65 — NIST security category 3, roughly equivalent to AES-192 — were selected over the lighter category-1 parameter sets (ML-KEM-512, ML-DSA-44) and the heavier category-5 sets (ML-KEM-1024, ML-DSA-87) as a middle-ground default consistent with the parameter choice reported in the closest comparable healthcare architecture we identified [5], and because category 3 is a commonly recommended default where a specific higher-assurance requirement has not been separately established. This is a

reasonable, evidence-consistent choice, not a claim that it is uniquely optimal — no source in this paper's literature review declares one specific PQC parameter set best for the EHR use case, and Section XV does not currently list a parameter-set sensitivity sweep as planned future work, though it would be a natural extension. ML-KEM and ML-DSA were selected as the KEM/signature pair over alternative NIST-selected or round-4 candidate schemes because they are the primary NIST-standardized choices [1], [2], pair naturally as a matched KEM/signature combination at the same security category, and appear as the PQC layer in the most directly comparable prior healthcare architecture [5] and in an evaluation targeting 6G contexts specifically [15].

B. Alternative Constructions Considered

Before settling on the construction in Section VII-C, this project's design phase considered several alternative combiner patterns, which we report here for completeness, with an explicit caveat about the confidence level behind each. A cascade or nested combiner, $K = \mathrm{KDF}_2(\mathrm{secret}_1, \mathrm{KDF}_1(\mathrm{secret}_2))$, was considered as an alternative to flat concatenation-then-extract-then-expand; it is structurally closer to how a layered construction combining a PQC-derived pre-shared key with an existing handshake protocol's own key schedule would typically operate, an approach broadly consistent with the layered design pattern reported by [11]. A deployed industry precedent for the flat-concatenation approach this paper ultimately uses — production hybrid post-quantum/classical TLS key exchange using a concatenation of a classical and a post-quantum shared secret, fed into TLS's existing key schedule — and a formal security analysis of concatenation-based hybrid KEM combiners in the general (non-QKD) cryptographic literature, both informed the decision to prefer flat concatenation over a cascade. We flag explicitly that our awareness of both of these specific results rests on general cryptographic background knowledge rather than independent primary-source verification performed for this paper, and we accordingly do not cite them as formal references here — consistent with the verification-tier discipline stated in Section III, we would rather name a consideration honestly at reduced confidence than cite a source we have not verified. The one concrete, structural correction this consideration process did produce, and that Section VII-C reflects, is moving the context-binding label out of the raw input keying material and into HKDF's dedicated `info` parameter — concatenating context directly into IKM is a subtly different, non-standard construction from using `info`/`salt` for domain separation as HKDF's own specification intends, and correcting this was a deliberate fix applied before any implementation existed, not an incidental detail.

C. Hybrid Key Derivation

The hybrid construction combines a QKD-derived secret K_{QKD} with an ML-KEM shared secret K_{PQC} via HKDF-Extract-then-Expand [16]:

$$IKM = K_{\mathrm{QKD}} \,\|\, K_{\mathrm{PQC}} \tag{1}$$

$$PRK = \text{HKDF-Extract}(salt, IKM) \tag{2}$$

$$K_{\text{session}} = \text{HKDF-Expand}(PRK, C_T, L) \tag{3}$$

where C_T is a context label binding the derived key to the specific session/transaction identifier and the negotiated mode, and $L = 32$ bytes. Binding the mode into C_T — rather than concatenating it into IKM — is a deliberate design correction: context belongs in the KDF's domain-separation input, not the raw keying material, and prevents a class of downgrade-confusion attack where a key derived under one mode could be misinterpreted as belonging to another. No independent salt source is modeled; $salt = \mathrm{null}$, which HKDF's specification treats as a string of zeros [16]. Concatenation order is fixed (K_{QKD} first); reversing it changes the derived key, confirmed against an independent reference implementation of Equations (1)–(3) during validation.

D. Explicit Security Claim Boundary

This construction is an **engineering-level key combination**, not a claimed instance of a formally proven compositional combiner. Standard KEM-combiner security proofs in the literature model both inputs as outputs of a key-encapsulation mechanism with a defined public key and encapsulation oracle; QKD-derived material is not a KEM output — it is a physical-layer symmetric string. Whether HKDF over a QKD secret and a KEM secret achieves the standard hybrid-combiner property (the session key remains secure if at least one input remains unbroken) is treated in this paper as a design assumption resting on the general hybrid-key-exchange literature, not independently re-derived or proven here. We state this claim boundary explicitly, in the same words wherever it recurs in this paper's supporting materials, specifically so it cannot silently drift into an overclaim in any downstream summary of this work. Alignment with formal composite-KEM standardization efforts currently under discussion in relevant standards bodies is a natural extension we did not pursue in the current implementation; we note it as future work (Section XV) rather than claim it.

E. Correctness Safeguard: Fail Loud, Not Fail Silent

Every ML-KEM establishment in this implementation performs an explicit consistency check that has no counterpart in a bare textbook description of KEM encapsulation/decapsulation: after the sender's `encap_secret` call produces (ciphertext, shared secret) and the receiver's `decap_secret` call recovers its own shared secret from that ciphertext, the two shared secrets are compared for equality *before* either is used to derive a session key. On any correctly functioning liboqs build this comparison always succeeds — a mismatch would indicate a broken cryptographic implementation, not a normal operational failure mode such as QKD unavailability or a network timeout, and none of the pilot's 757,500 transactions triggered it. The check exists anyway because the alternative — silently proceeding to derive

and use a session key from mismatched shared secrets — would produce two endpoints that believe they share a key but do not, a failure mode that is strictly worse than an explicit abort: it would surface later, non-deterministically, as an inexplicable decryption failure or, worse, not surface at all if an implementation bug happened to make both sides individually self-consistent. The implementation instead raises an explicit `EstablishmentFailure` exception and aborts the establishment attempt rather than returning a key. This is one instance of a general implementation-level design rule — correctness failures internal to the cryptographic layer fail loudly and immediately, rather than being caught, logged, and silently worked around — that is worth naming as a principle in its own right (Section VI-E), because it is easy for an efficient-looking exception handler added later to erode it without the erosion being obvious from the surrounding code.

F. Novelty Guardrails

Because a systems paper of this kind inevitably writes out its mechanisms in enough precision that a passage quoted out of context could read as claiming more novelty than is intended, we state explicitly what is and is not claimed as this paper's contribution, beyond the classification already given for C1–C3 in Section I. The hybrid key-derivation construction (Equations (1)–(3)) instantiates a known combiner pattern from the general hybrid-key-exchange literature; it is not presented, and must not be read, as a novel combiner construction — this is the single highest-risk passage in the paper for that misreading, which is why Section VII-D states its boundary immediately alongside it rather than only in a general limitations passage. The mode-synchronization handshake (Section V) is protocol plumbing specific to this system's integration of an already-known idea — availability-aware fallback, established in the power-grid domain by [4] and as a fail-safe layered pattern by [11] — into a new application context; it is standard state-confirmation engineering, not a cryptographic contribution. The key-rotation and pool-debit bookkeeping (Section IX) uses standard key-lifecycle-management concepts and should not be read as a novel key-management framework. The one element of this design most defensibly describable as a genuine, EHR-specific mechanism rather than a domain transplant is the criticality-aware branch in Algorithm 1 (the routine-versus-emergency wait/no-wait distinction), which has no counterpart in the power-grid mechanism this paper's evaluation methodology otherwise follows most closely — this is the paper's most defensible claim to a genuinely new mechanism, and we foreground it as such here rather than leaving it to be inferred.

VIII. Baselines

All five baselines share an identical interface, an identical AES-256-GCM payload-encryption step, and identical message framing, differing *only* in key-establishment, so that any measured difference between them is attributable to the key-establishment mechanism and not to an incidental implementation difference. Table IX defines them.

The distinction between B4 and B5 is the paper's central empirical claim and is enforced structurally, not just by convention: B4's implementation has no code path from a failed QKD draw to a PQC-only key — a QKD draw failure is an unconditional transaction failure. B5's implementation consults the adaptive controller first and only proceeds to PQC-only when instructed. B3's "QKD-only" label refers specifically to the session-key material; its classical control channel is authenticated with ML-DSA exactly as in every other baseline, so that baselines differ in key-establishment strategy rather than in authentication strength.

A. Why Five Baselines, Not Fewer

Each of the four comparison baselines exists to isolate one specific variable that a smaller baseline set would leave confounded with another, so we state each baseline's distinct methodological role explicitly rather than let it be inferred only from Table IX. B1 isolates the cost of quantum resistance itself: because it is the only baseline with no PQC component at all, the gap between B1 and B2 (both otherwise measured under identical conditions) is attributable specifically to PQC's own computational and communication cost, not to anything about QKD or adaptivity — without B1, a reader would have no baseline-level answer to "what does going post-quantum cost by itself," independent of this paper's QKD-specific contribution. B2 isolates the fallback target's own cost: because B5 falls back to exactly B2's key-establishment strategy under QKD unavailability, B2's measured latency and overhead (Table XVI) are, in effect, a direct empirical prediction of B5's own degraded-mode cost, which Section XI confirms holds for overhead exactly and for latency approximately, with the residual difference attributable specifically to the bounded-wait policy rather than to any discrepancy between the two baselines' cryptographic paths. B3 isolates the unmitigated failure mode: without a QKD-only baseline that is deliberately never allowed to fall back, there would be no clean reference point establishing what "QKD unavailability with no adaptive or hybrid mitigation at all" looks like on its own, as distinct from B4's specific failure to adapt *despite* also having a PQC component available. B4 isolates the adaptivity variable specifically: B4 and B5 are cryptographically identical in every respect except the presence of the controller in Algorithm 1 and its associated bounded-wait policy, which is precisely why the B4-versus-B5 comparison, and not any other pairing in Table IX, is this paper's central empirical claim (Section VII-F) — every other baseline pairing measures a different, secondary variable. A smaller baseline set — for instance, B2 and B5 alone — would still show that B5 tolerates QKD unavailability, but would not by itself demonstrate *why*: without B4 as a same-cryptography, no-adaptivity control, a critic could reasonably ask whether B5's resilience comes from its adaptive controller specifically or merely from some other, unstated implementation difference between it and a hybrid design. Holding every baseline to an identical interface, AEAD step, and message framing (Section VIII) is what makes this five-way decomposition methodologically sound rather than five superficially similar systems compared under uncontrolled conditions.

TABLE IX
THE FIVE EVALUATED BASELINES.

ID	Name	Key establishment	Behavior under QKD unavailability
B1	Classical	X25519 ECDH + Ed25519 signatures	Unaffected — does not use QKD (pre-quantum reference point; not part of the proposed architecture)
B2	PQC-only	ML-KEM-768 + ML-DSA-65	Unaffected — does not use QKD
B3	QKD-only	QKD pool draw (+ ML-DSA-authenticated control channel)	**Fails.** No fallback — the unmitigated-outage reference point
B4	Static hybrid	QKD ‖ ML-KEM via HKDF, always both	**Fails.** Does *not* fall back to PQC-only; demonstrates the single-point-of-failure risk of a non-adaptive hybrid
B5	Adaptive hybrid (proposed)	Controller (Algorithm 1) selects hybrid or PQC-only per session	Falls back to PQC-only and *succeeds*

B. Per-Baseline Protocol Detail

B1 (Classical). An ephemeral X25519 key pair is generated per session; the shared secret is derived via elliptic-curve Diffie-Hellman and used directly as AEAD key material (sized via HKDF where the raw shared-secret length does not already match the AEAD key size). Ed25519 provides endpoint authentication. This baseline is included specifically because it lacks post-quantum resistance — it is the pre-quantum reference point against which every other baseline's added cost is measured, not a candidate for deployment under the threat model of Section IV.

B2 (PQC-only). A receiver generates an ML-KEM-768 key pair; a sender encapsulates against the receiver's public key, producing a ciphertext and a shared secret at the sender, which the receiver recovers by decapsulation. ML-DSA-65 provides mutual authentication. This is the fallback target for B5 under QKD unavailability, and its measured cost (Table XVI) is therefore also, in effect, the cost B5 pays whenever it falls back.

B3 (QKD-only). The session key is drawn directly from the QKD pool (Section IX-D); no PQC key-encapsulation step occurs. The classical control channel is still ML-DSA-authenticated, since "QKD-only" in this evaluation refers to the session-key material specifically, not to the full authentication stack. `draw()` on an insufficient pool raises rather than returning any value, and the calling baseline propagates that as an unconditional transaction failure — this baseline never falls back to anything, by design, since it exists specifically as the unmitigated-outage reference point.

B4 (Static hybrid). Both a QKD pool draw and an ML-KEM encapsulation are attempted; the session key combines both via the construction in Section VII. If either input is unavailable — most consequentially, if the QKD draw fails — the transaction fails outright. No code path in this baseline's implementation can produce a PQC-only session key; this is the specific property Section XI confirms empirically, not merely asserts.

B5 (Adaptive hybrid). The adaptive controller (Algorithm 1) is consulted first. Regardless of the resulting mode, an authenticated mode-sync handshake announces the chosen mode to the receiving party before key-establishment proceeds — this is why B5's communication overhead (Section XI) includes one additional signature round trip beyond B4's, even though both baselines' PQC component is identical. If hybrid mode is selected, key-establishment proceeds exactly as in B4; if PQC-only is selected, it proceeds exactly as in B2. This baseline is the paper's proposed architecture.

TABLE X
VALIDATED ENVIRONMENT.

Item	Value
Platform	Ubuntu 24.04.4 LTS, x86-64
Python	3.12.3
liboqs (C library)	0.16.0, commit `8979276`, minimal build (ML-KEM-768, ML-DSA-65 only)
ML-KEM-768 sizes (measured)	pk 1184 B, ct 1088 B, shared secret 32 B
ML-DSA-65 sizes (measured)	pk 1952 B, signature 3309 B
Test suite	61/61 passing
End-to-end validation gate	8/8 checks passing

IX. SIMULATION IMPLEMENTATION

The system is implemented as a discrete-event simulation (using the SimPy library) orchestrating simulated device/client transaction arrivals for one experiment cell, where a cell is one specific combination of baseline, QKD availability level, device count, payload class, and network load. Every cryptographic operation executes for real: ML-KEM-768 and ML-DSA-65 through `liboqs` (a minimal build enabling only these two algorithms), AES-256-GCM and HKDF through a standard cryptography library. Table X records the validated execution environment.

All PQC parameter sizes reported in this paper are measured directly from the running library at call time via its own reported metadata, not recalled or assumed — a specific methodological correction from an earlier phase of this project, in which such sizes were initially drafted from training-data recollection and explicitly flagged as unverified until measured. The measured values coincide almost exactly with what had been recalled, which we note as a reassuring cross-check, not a substitute for having measured them.

A. Implementation Boundary

Precisely what is real code, what is a running simulation of modeled behavior, and what is excluded entirely was fixed at design time, before implementation began, to prevent scope from drifting silently during development. **Implemented as real code**: the adaptive decision logic (Algorithm 1); the key-management and pool model (Section IX-D); real PQC operations via `liboqs`, so that every latency and overhead number in Section XI is a genuine measurement, not a modeled estimate; real AEAD encryption and decryption of synthetic EHR payloads; and the simulation harness itself — topology, event scheduling, workload generation, baseline orchestration, and metrics collection. **Simulated as modeled behavior, executed as running code producing real output data, but not physically real**: the QKD subsystem's key-generation process (Section IX-D is a parameterized rate/pool model, not a real quantum channel); the 6G access and network layer (Section IX-E); and the overall multi-device topology and scaling behavior. **Modeled only abstractly, not simulated in fine-grained detail**: QKD physical-layer behavior (photon transmission, detector characteristics, error-correction and privacy-amplification internals) is abstracted to a rate/availability parameter, not simulated at the physical layer; 6G radio-layer protocol behavior (physical and medium-access layers) is abstracted to latency/throughput parameters, not modeled as an actual protocol stack; and hospital/EHR-server application logic beyond the security layer (clinical workflows, access-control policy) is out of scope and not modeled even abstractly, beyond the server existing, authenticating, and decrypting. **Not implemented at all**: physical QKD hardware, a quantum computer, real hospital infrastructure or real patient data, a production-grade hardened key-management system (the implementation is scoped to what the evaluation needs, not to a deployable KMS), and any novel cryptographic primitive.

A deliberate, explicitly acknowledged trade-off worth stating plainly: the QKD subsystem could in principle have been implemented as an actual protocol-level simulation (e.g., a BB84-style quantum-channel simulation using a quantum-computing simulation framework) rather than the parameterized rate/availability model this paper uses. A full protocol simulation would add physical-layer fidelity at real implementation and runtime cost. We judged a parameterized model sufficient for this paper's actual research question — adaptive behavior under varying QKD *availability*, not QKD physical-layer performance itself — and made that choice explicitly rather than defaulting into it silently. A reader evaluating whether this trade-off was the right one should weigh it against Section XIII's statement that this paper makes no claim about QKD physical-layer fidelity in the first place.

B. Test Suite

Table XI breaks down the 61-test regression suite by area. The integration-scenario tests specifically assert the B4/B5 divergence under outage that Section XI reports empirically at pilot scale; the bounded-wait tests were added specifically to catch the class of defect discussed in Section XII and are constructed so that at least one of them would fail against the pre-fix implementation.

TABLE XI
REGRESSION TEST SUITE BY AREA.

Area	Tests
QKD pool depletion, outage, recovery, no-fallback	9
HKDF extract-expand, context separation, length	7
ML-KEM / ML-DSA round trip, tamper rejection, sizes	7
Authentication (Ed25519 and ML-DSA)	3
Adaptive switching, thresholds, primitive isolation	6
B4/B5 divergence, PQC fallback, QKD-only failure	6
EHR generation, network delay, metric calculation	12
Bounded-wait path (recovery, timeout, determinism)	10
Total	**61**

1) liboqs Build Process: The `liboqs` Python wrapper's default installation attempts to auto-build the underlying C library with every algorithm liboqs supports, which is unnecessarily slow when only two algorithms are needed. This project instead built `liboqs` from source with a minimal algorithm selection:

```
cmake -GNinja -DCMAKE_BUILD_TYPE=Release
  -DOQS_MINIMAL_BUILD=
    "KEM_ml_kem_768;SIG_ml_dsa_65"
  -DOQS_BUILD_ONLY_LIB=ON
  -DBUILD_SHARED_LIBS=ON ..
ninja
```

producing a build in which *only* ML-KEM-768 and ML-DSA-65 are enabled — verified programmatically via the library's own enabled-algorithm listing at run time, not assumed from the build configuration alone. The implementation raises an explicit error at import time if either required algorithm is absent from the linked library, specifically to prevent a build problem from silently resulting in a different, unintended algorithm being substituted.

C. Key Management Lifecycle

Session keys are held in memory only, for the duration of their validity window; no persistent key storage is modeled, which is an appropriate simplification for a simulation study but is explicitly not modeled at the granularity a secure-enclave or hardware-security-module-backed deployment would require. Key rotation occurs at the earlier of a configured transaction count or elapsed time, whichever comes first, reusing the same window that determines the adaptive controller's per-session switching granularity (Section VI). An in-flight transaction using a key that is expiring is permitted to complete rather than being forcibly interrupted, a deliberate availability-over-strict-caution trade-off. On QKD outage recovery — once the pool refills above threshold — only the *next new session*

becomes eligible for hybrid mode again; recovery is evaluated at session boundaries, consistent with per-session switching granularity, and is not retroactively applied to a session already in flight under PQC-only mode.

D. QKD Pool Model

QKD is modeled as a bounded, replenishing resource, not as a quantum-mechanical process — no photon transmission, basis reconciliation, or quantum bit error rate distillation is simulated. The pool's level L evolves as

$$L \leftarrow \min(C,\ L + r\,\Delta t) \tag{4}$$

where C is capacity in bits and r is the generation rate in bits/second, advanced by elapsed simulated time Δt. Availability is the normalized fraction $a = L/C \in [0, 1]$, which is exactly the input Algorithm 1 consumes. An outage suspends generation ($r_{\text{effective}} = 0$) while draws continue to deplete the pool, matching the intended failure semantics of a real channel interruption. Pool capacity is sized relative to a session's key draw (20 sessions' worth at 256 bits/session, giving $C = 5{,}120$ bits by default) rather than an arbitrary constant, following a mid-implementation recalibration finding worth reporting in full because it illustrates a general risk in simulation-based evaluation. An earlier version of this implementation fixed pool capacity at a large constant (8,000,000 bits) with the generation rate scaled to that same constant. Under that configuration, at 20 simulated devices generating 200 transactions over a short simulated window, every single transaction resolved to hybrid mode regardless of the configured availability level — the independent variable this entire study exists to sweep had no observable effect, because the pool never drained meaningfully relative to a 256-bit session draw. The fix re-anchored capacity to a session's draw size and the full-availability generation rate to "regenerate the whole pool in approximately one simulated second." Re-validated at the same 20-device, 200-transaction scale after the fix: at $a = 1.0$, 200/200 transactions resolved to hybrid; at $a = 0.5$, 114/200 hybrid and 86/200 fallback; at $a = 0.0$, 15/200 hybrid (drawn from the pool's initial charge before it drained) and 185/200 fallback. Only after this recalibration did the availability parameter have the real, monotonic, visible effect the pilot in Section XI reports. We report this specific failure mode because it is a general one: a parameter that appears in a configuration file and is swept across an experiment matrix is not thereby guaranteed to have a measurable effect on the system under test, and confirming that it does is a necessary validation step, not an optional one. All QKD pool parameters — capacity, generation rate, and the mapping from a configured "availability level" label to a concrete generation rate — are modeled assumptions and explicit sensitivity variables, not literature-measured facts; the deployed 300 km trusted-node network reported in [3] is used only as loose plausibility context for these choices, not as their source.

E. 6G-Edge Network Model

The network is modeled as three sequential hops — EHR/IoMT client to a simulated 6G access abstraction, to an edge gateway, to the hospital backbone and EHR server — each characterized by configurable propagation delay, processing delay, transmission rate, and per-hop packet loss probability. Per-hop latency for a payload of B bits is

$$\ell_{\text{hop}} = d_{\text{prop}} + d_{\text{proc}} + \frac{B}{\rho} \tag{5}$$

where d_{prop} is propagation delay, d_{proc} is processing delay, and ρ is the hop's transmission rate; end-to-end network latency is the sum of Equation (5) over all three hops. Packet loss is modeled as an independent per-hop Bernoulli event at a configured probability; a loss on any hop fails the transaction, with no retransmission or queueing modeled. Two named parameter profiles are used throughout this paper: a *nominal* profile (low per-hop loss probability, used for every pilot cell reported in Section XI) and a *congested* profile (higher loss probability, used only in the reproducibility verification described in Appendix B, not in the pilot itself). This is a deliberately simplified abstraction: it does not implement any real 6G protocol stack (no physical layer, no medium-access layer, no radio resource management), and 6G standards themselves are not finalized. In the pilot reported here, all three hops share one identical parameter set per profile; Section XIII identifies this as a limitation, since it flattens the access-versus-backbone distinction a 6G-edge topology would realistically have — a real deployment's radio access hop and its wired backbone hop would not share a single loss and rate profile.

F. EHR Workload Model

The EHR workload generator produces synthetic, FHIR-inspired transaction bodies; no real patient data exists anywhere in this project, and a dedicated regression test asserts the absence of real-record-shaped fields in every generated body. FHIR resource shapes inform the generator's structure for realism, without claiming FHIR-standard conformance — conformance itself is not this paper's research question and pursuing it would be a distraction from it. Payload size is configurable by class: small (1–5 KB, representative of a single vitals or observation update), medium (20–80 KB, representative of a visit summary or structured problem/medication list), and large (200 KB–1 MB, representative of a multi-document bundle combining history, labs, and imaging references); the pilot reported here used the medium class exclusively across all 30 configurations, so payload-class sensitivity is explicitly not evaluated by this pilot (Section XIII). Three transaction types are modeled — read, write, and share — of which *share* is used exclusively in the pilot's transaction generation, as the type most directly matching this paper's EHR-sharing framing; read and write are defined in the generator's structure but not exercised in the results reported here. A configurable fraction of transactions (5% by default) are flagged emergency-criticality, which is the input Algorithm 1 uses to skip the bounded wait; criticality is a property applicable to a transaction of any size class, not a separate size class itself. Generation is procedural and deterministic under a given seed, avoiding both patient-privacy overhead and the uncontrolled payload-characteristic variability a real dataset would introduce, which would confound isolating the adaptive

mechanism's own effect — the specific property this pilot's controlled generation is designed to preserve.

G. Metric Definitions

Table XII gives the formal definition of every metric this paper reports or is designed to report, specifying for each its numerator, denominator, measurement point, unit, and aggregation method. Not every metric defined here is reported with results in Section XI: throughput and CPU/memory cost were defined at design time but not instrumented in the pilot's implementation, and their absence is recorded as a limitation (Section XIII) rather than silently omitted.

H. Anticipated Risks and What Actually Happened

Before any code existed, this project's architecture specification explicitly flagged a category of implementation risk worth reporting here because it materialized almost exactly as anticipated. The specification identified the mode-synchronization and adaptive decision-logic path as "the most protocol-complexity-dense part of this design and the most likely place for a subtle bug," warning specifically that such a bug could "happen to 'work' by always defaulting to one mode without the adaptive mechanism actually being exercised" — and that this specific failure mode would threaten the validity of the fallback-frequency metric and Hypothesis H1 (Section XII) precisely because the system could appear adaptive in its logs while not actually being driven by real QKD-availability state. This is, in essence, an exact prospective description of the wait-path defect documented in Section XII: the pre-fix implementation recorded 'waited=True' for the correct configured duration on every degraded-band transaction while the wait itself never executed, producing exactly the "looks adaptive, isn't" failure mode the specification had warned against before implementation began. We report this correspondence not to claim prescience, but because it illustrates that the risk was a foreseeable category of defect, not an idiosyncratic accident — and that foreseeing a risk category in a design document is not, by itself, sufficient to prevent the specific instance from occurring; only a test constructed to fail against the unfixed behavior (Section XII) closed the gap. The same design document also flagged a general "fair-baseline risk" — that implementation shortcuts could cause the five baselines to diverge on something other than key-establishment strategy, contaminating every comparison in Section XI — which motivated the structural fairness property described in Section VIII (identical AEAD, identical framing, differing only in key-establishment) being enforced in the shared implementation rather than left to per-baseline discipline.

I. Metrics and Reproducibility

Every transaction produces one raw JSON-lines record before any aggregation occurs, carrying its full identifying context (experiment identifier, baseline, configured QKD availability, device count, payload class, network load, seed) together with success/failure, key-establishment latency, payload-encryption latency, network latency, end-to-end latency, communication overhead in bytes, the key source actually used, the controller state, and the failure reason where applicable. Per-cell summary statistics (mean, median, 95th percentile, and bootstrap 95% confidence intervals for latency, which is expected to be right-skewed by the controller's bounded-wait behavior) are computed from these raw records, never the reverse. Reproducibility rests on one top-level seed, from which a per-component seed manager derives independent, named sub-seeds so that, for example, adding a workload draw cannot perturb the network layer's independent random sequence; every experimental run additionally records its own software-environment provenance.

X. Experimental Methodology

Consistent with the paper's own hypotheses (Section XII), the evaluation reported here is a *pilot*: its explicit purpose is to validate that the adaptive mechanism behaves correctly and to produce a variance estimate that would inform the repetition count and parameter ranges of a subsequent full study, not to be the final word on the trade-off surface. The pilot matrix sweeps 3 QKD availability levels ($a \in \{0.0, 0.5, 1.0\}$) $\times$ 2 device counts ($\{10, 1{,}000\}$) $\times$ 1 payload class (medium) $\times$ 1 network load (nominal) $\times$ 5 baselines, for 30 configurations, each repeated 5 times with independently derived seeds, for 150 total simulation runs and 757,500 simulated transactions. Adaptive-controller thresholds were fixed at $\tau_{\text{hybrid}} = 0.3$, $\tau_{\text{wait}} = 0.1$, and $T_{\text{wait}} = 0.05\,\text{s}$ throughout the pilot; these are provisional values, not independently validated as optimal, and characterizing their effect on the trade-off surface is future work.

A. Parameter Provenance

Every numeric parameter this simulation uses falls into one of three categories, summarized in Table XIII: measured directly from a real library at run time, internally derived from another parameter, or a modeled assumption chosen for tractability and flagged as a candidate for future sensitivity analysis. No parameter in this paper is presented as a literature-measured fact unless Table XIII says so explicitly.

B. Expected Trade-offs, Stated Before Results

Prior to running the pilot, this project's design phase recorded the following as testable expectations, not results, specifically to guard against post-hoc reinterpretation of whatever the pilot produced. We report them here, alongside what the pilot in Section XI actually showed, so the two can be compared directly. Higher QKD availability was expected to reduce reliance on the PQC-only fallback path, but QKD's own contribution was expected to add key-establishment latency relative to PQC-only, since QKD's practical key-generation rate was expected to be a binding constraint relative to a purely computational PQC operation — this was *not* confirmed by the pilot: at full availability, B5's mean latency (0.43 ms) is close to B4's, not meaningfully elevated by the QKD contribution, because the pool draw itself is fast when the

TABLE XII
FORMAL METRIC DEFINITIONS.

Metric	Numerator / Denominator	Measurement point	Unit	Aggregation
key-establishment latency	(session key derived + mode-sync confirmed) − (establishment initiated)	Edge Gateway	ms	mean, median, p95
End-to-end EHR latency	(ack received at client) − (transaction initiated)	EHR client	ms	mean, median, p95; reported separately by criticality where instrumented
Communication overhead	Total bytes transmitted for key-establishment (handshake/auth messages); also reported as a ratio to payload bytes	Summed across path	bytes; dimensionless ratio	mean per transaction, per baseline
Successful-transmission rate	Successful transactions / total attempted, per cell (not globally averaged, so degradation patterns are not obscured)	End of transaction lifecycle	%	per cell
Throughput†	Successfully completed transactions / simulated wall-clock window	System-wide, per cell	transactions/s	per cell, per baseline
Fallback frequency	Sessions resolving to PQC-only / total sessions in cell (B5 only)	At session establishment	%	per cell, vs. QKD availability
Scalability	Trend of latency, success rate, and throughput vs. device count, not a single number	Inherited from underlying metric	—	degradation curve
CPU/memory cost†	CPU time / peak memory of cryptographic operations specifically, not the whole simulation process	Instrumented around PQC/QKD library calls	ms; MB	mean per operation type

†Defined at design time; not instrumented in the pilot reported in this paper (Section XIII).

TABLE XIII
PARAMETER PROVENANCE.

Parameter	Value	Type	Notes
ML-KEM-768 / ML-DSA-65 sizes	Table X	**Measured** (real liboqs)	Superseded earlier recalled-but-unverified figures
QKD bits drawn per session	256	Internally derived	Matches AES-256 key length
QKD pool capacity	sessions_buffer × 256 (5,120 bits default)	Modeled assumption	Section IX-D's recalibration; sensitivity-testable
QKD generation-rate mapping	Linear in a; full rate $= C/1\,\mathrm{s}$	Modeled assumption	Explicit sensitivity variable
Adaptive thresholds ($\tau_{\text{hybrid}}, \tau_{\text{wait}}, T_{\text{wait}}$)	0.3, 0.1, 0.05 s (pilot default)	Modeled assumption	Not literature-sourced; not independently validated as optimal
EHR payload size ranges	Small 1–5 KB; medium 20–80 KB; large 200 KB–1 MB	Modeled assumption	Unchanged from early design phase; not FHIR-measured
Network latency/throughput/loss (nominal, congested)	Section IX-E	Modeled assumption	Concrete values now exist in code where design phase specified only bracketed ranges

pool is not depleted; the large latency increase observed under degraded availability (Section XI) is the bounded-wait policy, not the QKD draw operation itself. Adaptive fallback (B5) was expected to improve successful-transmission rate relative to QKD-only (B3) and possibly static hybrid (B4) specifically under degraded availability, without being expected to strictly dominate B4 in every condition — this was confirmed, and the margin (99.68% vs. 0.20% at $a = 0.0$) was larger than a cautious pre-registered expectation would have predicted. Fallback frequency was expected to increase with both decreasing availability and increasing device count, a compounding effect between two independent variables — this was confirmed (Section XII). Communication overhead was expected to be highest for B4 and lowest for B1/B2, with B5 tracking close to B2 under low availability and closer to B4 under high availability, essentially by construction of the adaptive policy — this expectation was *not* confirmed in the form stated: B5's overhead (Section XI) is constant at 8,890 bytes regardless of mode, because the mode-sync handshake cost is paid unconditionally, which the pre-registered expectation had not anticipated as a fixed rather than mode-dependent cost.

C. Computational Cost

Because every cryptographic operation in this simulation executes for real (Section IX-A) rather than being modeled analytically, the pilot's wall-clock cost is itself worth reporting, both for reproducibility planning and because it varies meaningfully by baseline. Table XIV reports measured wall-clock time for one full cell at the pilot's largest scale (1,000

TABLE XIV
MEASURED WALL-CLOCK TIME, ONE CELL AT 1,000 DEVICES / 10,000 TRANSACTIONS, BY BASELINE.

Baseline	Wall-clock time (s)	Per-transaction (μs)
B1 (Classical)	6.42	642
B2 (PQC-only)	5.74	574
B3 (QKD-only)	3.30	330
B4 (Static hybrid)	4.02	402
B5 (Adaptive hybrid)	7.79	779

TABLE XV
CONTROLLED VS. INDEPENDENT VARIABLES.

Controlled (fixed)	Independent (varied)
PQC algorithm (ML-KEM-768/ML-DSA-65)	QKD availability level (a) — primary
Simulated compute profile (host CPU throughout)	Device/user count — scalability axis
Network topology structure (3-hop)	
Target security level (NIST category 3)	
Payload class (medium) and network load (nominal) in this pilot	Payload class and network load in a full study (future work)

devices, 10,000 transactions), by baseline, on the environment in Table X. B5 is the most expensive baseline per transaction, consistent with it performing the union of B4's cryptographic work plus an additional mode-sync signature round trip; B3 is the least expensive, since it performs no PQC key-encapsulation operation at all. The full 150-run pilot completed in several minutes of wall-clock time on this environment; a full study sweeping a substantially larger configuration matrix (Section XV) should budget runtime proportionally, and, per Section XIII's implementation-risk discussion, estimating this cost from a small subset before committing to a full matrix is a deliberate practice this project followed, not an afterthought.

D. Controlled and Independent Variables

Table XV distinguishes the variables this pilot holds fixed to isolate the effect under study from those it deliberately varies. Holding the PQC algorithm choice, target security level, and network topology structure fixed within a given comparison is what makes a difference between baselines attributable to key-establishment strategy rather than to an incidental configuration difference; QKD availability is the primary independent variable, since it is the condition the adaptive mechanism exists to respond to, and device count is the secondary axis this pilot uses to test whether the mechanism's behavior holds at different simulated load levels.

XI. RESULTS

All results below are pooled across the 5 repetitions of each of the 30 configurations, from the raw JSON-lines transaction records. No between-baseline hypothesis test has been applied; every number reported is a descriptive statistic, and every confidence interval is a bootstrap interval over the pooled sample for that cell.

A. Statistical Methodology

For each cell, latency percentiles are computed by linear interpolation between order statistics: for a sorted sample of n values and target percentile p, the rank $k = (n-1) \cdot p/100$ is computed, and the result interpolates linearly between the values at $\lfloor k \rfloor$ and $\lceil k \rceil$. Ninety-five percent confidence intervals for mean key-establishment latency are computed by bootstrap resampling of the mean (1,000 resamples per cell, with replacement, at the pooled cell's sample size), rather than by a normal-approximation interval. This choice is deliberate: the adaptive controller's bounded-wait behavior (Algorithm 1) is expected to produce a right-skewed latency distribution — a transaction either resolves quickly (no wait) or is delayed by close to the full wait timeout — for which a normal-distribution confidence interval would misstate the true interval. Fallback frequency for B5 is computed as the fraction of B5 sessions whose recorded key source is the PQC-only path, over all B5 sessions in the cell (both successful and failed).

B. Figure Design Notes

The three figures in this section follow a consistent design discipline worth stating explicitly, since the choices are deliberate rather than default plotting-library behavior. Figure 4 initially plotted all five baselines with one hue each, which we abandoned after inspection: B1, B2, B3-at-full-availability, and B5 sit at numerically near-identical values, and B3 and B4 trace nearly identical collapse curves, so five distinctly colored lines rendered as only two visually distinguishable paths, with the paper's actual central comparison (B4 versus B5) no more visually prominent than any other pairing. We replaced this with an emphasis treatment — B4 and B5 in distinct, saturated colors, B1–B3 in a muted gray as context — specifically because the underlying data, not an arbitrary stylistic preference, made a uniform categorical treatment misleading about where the paper's actual finding lives. Figure 5 uses a logarithmic latency axis because the underlying range spans roughly two orders of magnitude (Section XI); a linear axis would compress the full-availability values into visual indistinguishability. Figure 6 uses a single sequential hue across all five bars, rather than five categorical colors, because it encodes a pure magnitude comparison (bytes by baseline) with no second identity dimension for color to carry — a categorical palette there would imply a distinction the data does not have.

C. Successful Transmission Rate

Figure 4 shows successful-transmission rate against QKD availability, faceted by device count. B1, B2, and B5 sustain approximately 99.7% across every condition; the residual failures at that level are attributable to nominal-load network packet loss, not key-establishment failure. B3 and B4 collapse together as availability falls: at $a = 0.0$ and 1,000 devices,

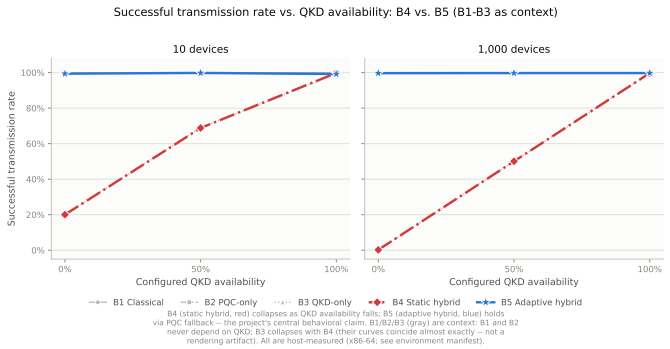

Fig. 4. Successful transmission rate vs. QKD availability. B4 (red) and B5 (blue) are emphasized as the paper's central comparison; B1–B3 (gray) are context and, for B1/B2/B5-at-full-availability and for B3/B4, are numerically near-coincident (see caption in figure).

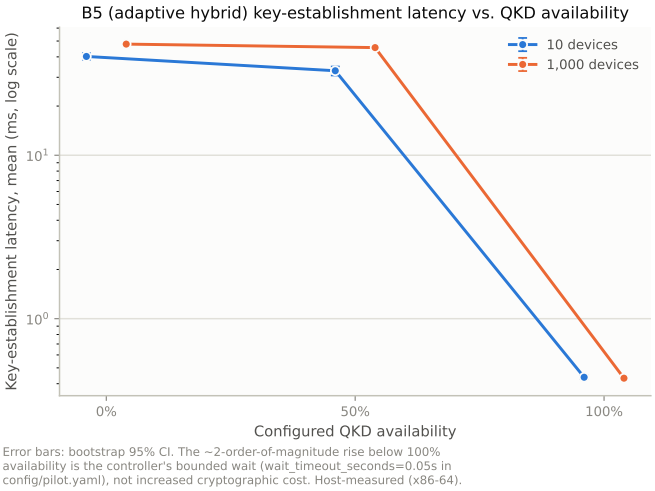

Fig. 5. B5 key-establishment latency (mean, bootstrap 95% CI, log scale) vs. QKD availability, by device count.

B3 achieves 100/50,000 successful transmissions (0.20%) and B4 achieves 100/50,000 (0.20%) — indistinguishable to three significant figures despite using entirely different key material. B5, under the identical condition, achieves 49,842/50,000 (99.68%). At the intermediate availability level ($a = 0.5$, 1,000 devices), B3 reaches 25,018/50,000 (50.04%) and B4 reaches 25,033/50,000 (50.07%); B5 reaches 49,862/50,000 (99.72%).

D. Key-Establishment Latency

Figure 5 shows B5's mean key-establishment latency against QKD availability, with bootstrap 95% confidence intervals, faceted by device count. At full availability, mean latency is 0.4380 ms (10 devices) and 0.4323 ms (1,000 devices) — comparable to B4's 0.2556 ms/0.2626 ms plus the additional mode-sync signature round trip B5 always performs. At $a = 0.5$, mean latency rises to 32.89 ms (10 devices) and 45.58 ms (1,000 devices); at $a = 0.0$, to 40.24 ms and 47.88 ms respectively — a rise of roughly two orders of magnitude, and close to the 50 ms bounded-wait timeout configured for the pilot. This is the controller's bounded wait for QKD pool replenishment, not increased cryptographic cost: the underlying ML-KEM and ML-DSA operations measure in the tenths of a millisecond regardless of QKD state (Table X). Correspondingly, the fraction of B5 sessions resolving to the PQC-only fallback path rises with both decreasing availability and increasing device count: from 0.0% ($a = 1.0$) to 3.2–4.7% ($a = 0.5$, 10 and 1,000 devices respectively) to 85.0–99.85% ($a = 0.0$, 10 and 1,000 devices respectively). The device-count dependence at fixed availability reflects a fixed-size pool draining faster under higher simulated load, spending a larger share of the observation window in the degraded or unavailable band.

E. Latency Distribution Shape

The mean latency values reported above obscure a distributional detail worth surfacing on its own, because it is the clearest empirical confirmation of the bimodal behavior Algorithm 1 is expected to produce (Section VI-B). At the 1,000-device configuration and $a = 0.0$, mean B5 key-establishment latency is 47.877 ms, but the *median* is 50.384 ms and the 95th percentile is 50.629 ms — the median exceeds the mean, and the top five percent of the distribution sits barely above the median at all. This pattern is the signature of a distribution concentrated at the 50 ms wait-timeout ceiling with a smaller, faster-resolving minority pulling the mean downward, rather than a smooth unimodal spread around the mean: at $a = 0.0$, the large majority of routine transactions exhaust the full wait budget before falling back (consistent with the 99.85% fallback frequency reported in Table XVI), while emergency transactions and the residual routine transactions whose pool crosses threshold quickly contribute the faster tail that separates the mean from the median. The same pattern holds at $a = 0.5$ (median 50.387 ms, p95 50.626 ms, against a mean of 45.583 ms), but not at $a = 1.0$, where median (0.407 ms) and p95 (0.658 ms) both sit below the mean's own order of magnitude, consistent with the near-total absence of waiting once the pool is abundant. End-to-end latency (key-establishment plus payload encryption plus simulated network transit) shows the identical qualitative shape shifted by a roughly constant 11.4 ms network-transit offset: end-to-end median at $a = 0.0$ is 61.785 ms against a p95 of 63.160 ms, a narrow spread consistent with the network-transit component itself contributing comparatively little variance next to the wait-dominated key-establishment component. Payload AES-256-GCM encryption cost (Section IX) is essentially constant across every condition, 0.036 ms–0.043 ms regardless of baseline, availability, or device count, confirming that none of the latency variation reported in this section originates in the payload-encryption path added by the M3 instrumentation fix (Section XII) — it is entirely attributable to key-establishment, as the architecture predicts.

F. Communication Overhead

Figure 6 reports mean key-establishment communication overhead by baseline, constant across every QKD availability level and device count in the pilot. Every observed value matches its value computed by hand from the measured

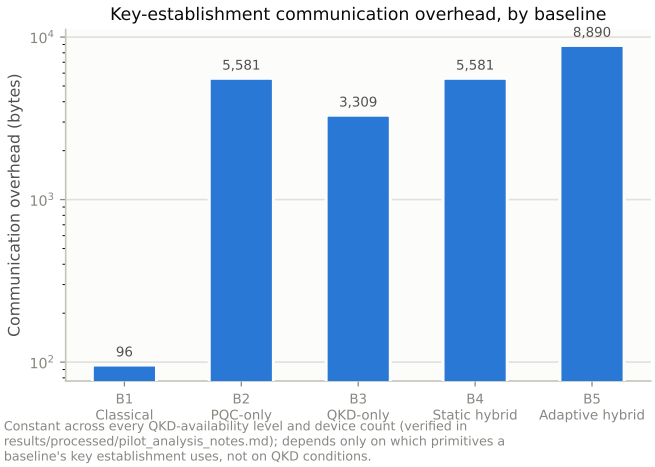

Fig. 6. key-establishment communication overhead by baseline (log scale).

primitive sizes in Table X exactly: B1, 96 bytes (X25519 public key 32 B + Ed25519 signature 64 B); B2, 5,581 B (ML-KEM public key + ciphertext, 2,272 B, plus ML-DSA signature, 3,309 B); B3, 3,309 B (ML-DSA signature only; QKD exchange bytes are not counted toward this metric by design); B4, 5,581 B (identical PQC component to B2); B5, 8,890 B (the PQC component, 2,272 B, plus two signature round trips — mode-sync and session establishment — at 3,309 B each). B5's overhead is identical whether it resolves to hybrid or PQC-only mode, since both paths perform the same PQC exchange and the same two signature round trips; only the QKD draw differs between them, and QKD material is not counted toward this metric, matching B4's convention. This exact agreement between observed and hand-computed overhead served as an internal consistency check on the measurement pipeline itself, not as a novel finding.

G. Summary Table

Table XVI reports the full pooled summary for the 1,000-device configuration across all three availability levels and all five baselines, the cell size at which the paper's headline claims are drawn.

XII. Discussion

A. Contribution Fulfillment

Table XVII maps the three contributions claimed in Section I against what this paper actually delivers, so a reader can check the claim against the evidence directly rather than take the framing in Section I on faith.

B. Ethical Considerations

No real patient data, human subjects, or clinical systems were used or involved at any point in this project. The EHR workload (Section IX-F) is entirely synthetic and procedurally generated; a dedicated regression test asserts the absence of real-record-shaped fields in every generated transaction body, checked as part of the test suite reported in Section IX-B. No institutional review was required or sought, consistent with this being a systems and cryptographic-performance evaluation rather than a study involving people, records, or clinical outcomes — a distinction Section XIII states explicitly is also a scope boundary: this paper makes no clinical-outcome or deployment-readiness claim of any kind.

C. Hypothesis Evaluation

H1 (availability/resilience) predicted that the adaptive mechanism would degrade gracefully toward PQC-only-level performance as QKD availability decreased, while the static hybrid baseline would show a sharper drop at the same availability levels. The results support this: B5's successful-transmission rate stays within measurement noise of B2's (PQC-only) rate at every availability level, while B4 collapses to a rate statistically indistinguishable from B3's unmitigated QKD-only baseline. The word "sharper" understates what was observed — B4 does not degrade gracefully at all; it fails almost totally at $a = 0.0$, which is the mechanism-level distinction this paper set out to demonstrate.

H2 (EHR-specific cost) predicted that adaptive-switching latency and overhead would remain within acceptable bounds for routine access but might exceed an emergency-access latency budget specifically. The results are consistent with the qualitative shape of this prediction but the pilot as designed cannot fully test it: no explicit emergency-access latency budget was defined or evaluated against in this pilot, so we can report the measured latency (Section XI) but not confirm or refute the budget-violation claim as stated. We flag this as an incomplete test of H2, not a confirmed result, and identify defining an explicit budget as necessary future work (Section XV).

H3 (scalability) predicted that fallback frequency would scale with device count in a pattern broadly consistent with [4]'s power-grid findings, with different numeric thresholds. The observed device-count dependence (fallback frequency rising from 85.0% to 99.85% between 10 and 1,000 devices at $a = 0.0$, and from 3.2% to 4.7% at $a = 0.5$) is directionally consistent with this prediction: a fixed-capacity pool drains faster under higher simulated concurrent load relative to its regeneration rate, so a larger fraction of the observation window falls into the degraded or unavailable band as device count increases. This is a mechanical consequence of Equation (4) rather than an emergent property specific to this workload, which is itself worth noting: the same qualitative scaling relationship should be expected from any fixed-capacity, fixed-rate pool model regardless of the traffic riding on top of it, which is consistent with why [4]'s power-grid evaluation reports a similar qualitative pattern despite an entirely different traffic class. We did not, however, obtain the specific numeric thresholds reported in [4] at sufficient confidence (that source is cited at the partial-verification tier described in Section III) to perform the direct numeric comparison C3 originally specified; this remains open, and Section XV identifies obtaining that source at full-verification confidence as a specific, addressable next step rather than a permanently closed question.

TABLE XVI
POOLED SUMMARY, 1,000-DEVICE CONFIGURATION, $n = 50{,}000$ PER CELL (5 REPETITIONS $\times$ 10,000 TRANSACTIONS).

Baseline	a	Success rate	key-establishment mean (ms)	Overhead (B)	Fallback frequency
B1	0.0	0.9970	0.2961	96	—
B1	0.5	0.9971	0.2985	96	—
B1	1.0	0.9972	0.2932	96	—
B2	0.0	0.9968	0.2422	5,581	—
B2	0.5	0.9970	0.2468	5,581	—
B2	1.0	0.9970	0.2429	5,581	—
B3	0.0	0.0020	0.1852	3,309	—
B3	0.5	0.5004	0.1733	3,309	—
B3	1.0	0.9968	0.1665	3,309	—
B4	0.0	0.0020	0.2618	5,581	—
B4	0.5	0.5007	0.2887	5,581	—
B4	1.0	0.9970	0.2626	5,581	—
B5	0.0	0.9968	47.877	8,890	0.9985
B5	0.5	0.9972	45.583	8,890	0.0470
B5	1.0	0.9973	0.4323	8,890	0.0000

TABLE XVII
CONTRIBUTION FULFILLMENT.

#	Claimed	Delivered
C1	Adaptive QKD-PQC key-establishment architecture for EHR sharing in a simulated 6G-edge topology	Section V (components, data flow), Section VI (decision logic, Algorithm 1); implemented as real, tested code (Section IX-B), not only specified
C2	Multi-metric evaluation against four baselines under varying QKD availability and device count, on a synthetic EHR workload	Section XI and Table XXI, all 30 configurations, 757,500 transactions; five metrics reported (success rate, key-establishment latency, end-to-end latency, overhead, fallback frequency) of the eight formally defined (Table XII)
C3	Characterization of where EHR-relevant latency budgets are/aren't preserved under degraded QKD availability, compared numerically against general-infrastructure thresholds from prior work	**Partially delivered.** Latency is characterized precisely (Section XI); the direct numeric comparison against [4]'s thresholds could not be completed at full-verification confidence (Section XII), and no explicit emergency-access latency budget was defined against which to test preservation — both are named, specific future work (Section XV), not silently dropped

D. Comparison Against an Oracle Policy

It is worth stating explicitly what an idealized, unrealizable policy would achieve, as a ceiling against which B5's real, measured behavior can be read. An oracle policy — one with perfect, instantaneous, zero-cost knowledge of true QKD pool state at the moment of every key-establishment decision, and no bounded-wait budget constraint — would always select HYBRID whenever the pool holds enough material for the current session and PQCONLY otherwise, with zero decision latency and zero wasted wait time. Such a policy is unrealizable in this architecture because pool state can only be queried, not know a priori, and a real decision incurs at least the cost of that query; it exists here purely as an analytical reference point, not as a sixth baseline evaluated empirically. Two gaps between B5 and this oracle are visible directly in the results already reported. First, the oracle would never wait — it would know immediately whether the pool will cross threshold before the bounded-wait window elapses, and would only ever pay the wait cost when it resolves in HYBRID's favor, never when it resolves in PQCONLY's. B5, by contrast, pays a meaningful share of its bounded-wait budget on transactions that ultimately fall back anyway (Section XI's finding that median latency at $a \in \{0.0, 0.5\}$ sits near the 50 ms ceiling regardless of eventual outcome) — this wasted wait is the concrete, measurable price of not having oracle knowledge, and it is the single largest component of B5's latency cost relative to B4 under healthy availability. Second, the oracle's successful-transmission rate would be strictly bounded by the same physical pool-exhaustion limit B5 faces at $a = 0.0$ — an oracle cannot manufacture QKD key material that does not exist, so an oracle policy at $a = 0.0$ would also fall back to PQC-only on essentially every transaction, achieving a successful-transmission rate close to B5's own 99.68%, not close to 100%. This second point is the more important one for interpreting this paper's central claim correctly: B5's 99.68%-versus-B4's-0.20% gap at $a = 0.0$ is not a gap between B5 and an achievable ideal, it is a gap between an adaptive policy that behaves close to the physically achievable ceiling and a non-adaptive policy that does not adapt at all. The oracle comparison therefore sharpens, rather than undercuts, this paper's central finding: the room for a smarter real-world policy to improve on B5 is bounded mainly by the wasted-wait gap identified above, on the order of tens of milliseconds per degraded transaction, not by the successful-transmission-rate gap that this paper's headline result is built on.

E. Go/No-Go Assessment for Full-Study Expansion

Before this pilot was run, this project's methodology fixed six explicit, checkable criteria that must *all* hold before expanding to a larger study — deliberately checkable rather than an "if it looks good" judgment. We evaluate each against the pilot's actual results in Table XVIII.

Five of six criteria are met without qualification; the sixth (consistency) is met at the level the criterion actually specifies — mode agreement, not latency agreement — but we flag the latency divergence explicitly rather than letting a satisfied checkbox imply more than it does. Per this project's own pre-registered decision rule, all six criteria being satisfied means expansion to a larger study is methodologically justified; consistent with that same rule, we do not treat this as license to restore an arbitrarily large configuration matrix, but as justification to size the next study against this pilot's actual runtime and variance data (Section XV), not against an earlier, undata-driven design target.

F. Interpreting the Overhead Result

The exact agreement between B5's measured 8,890-byte overhead and its value computed by hand from Table X's measured primitive sizes (Section XI) is worth a further remark beyond its role as an internal consistency check. It confirms, independent of any claim this paper makes about the adaptive mechanism's behavior, that B5's communication cost is entirely determined by which cryptographic operations it performs and not by which mode it resolves to — the mode-sync handshake that makes B5's overhead exceed B4's by exactly one ML-DSA signature (3,309 bytes) is paid whether the session ultimately uses QKD material or not. A deployment evaluating whether to adopt the adaptive mechanism over the static hybrid baseline can therefore treat B5's communication-overhead cost as a fixed, mode-independent tax for the resilience shown in Table XVI, separate from the latency cost discussed above, which *is* strongly mode-dependent.

Table XIX summarizes all three hypotheses' evaluation status.

G. The Instrumentation Fix: Why This Matters for Trusting the Results Above

The results in Section XI depend on a specific correction made during this project's validation phase, and we report it because a reader evaluating these results should know it happened. The adaptive controller's bounded-wait step (Algorithm 1, line 8) is implemented as a function call that the calling baseline must explicitly wire to a time-advancing function. In an earlier version of this implementation, that wiring was absent: the wait step was a no-op, so every transaction in the controller's degraded band fell through immediately to PQC-only, while the recorded decision metadata nonetheless reported that a wait had occurred, for the full configured duration. This was silent — no error, no exception, no failing test, because the controller's own unit tests exercised the wait path correctly by injecting a wait function directly, and simply never caught that the calling code did not do the same.

The fix was constrained by the simulation's own architecture in a way worth explaining, because the obvious fix was not available. The natural implementation of a bounded wait in a discrete-event simulation is to yield control back to the event scheduler for the wait duration; the simulation's discrete-event scheduler supports exactly this pattern for the process that drives each simulated device. However, key-establishment itself is implemented as an ordinary synchronous function call, not as a scheduler-aware process — only the outer per-device loop that repeatedly calls it is. Making the wait yield to the scheduler correctly would therefore have required restructuring every baseline's key-establishment path into a scheduler-aware form, a substantially larger change than the defect warranted. The fix instead exploits a property already present in the QKD pool model: the pool carries its own simulated clock, advanced by explicit time-elapsed calls that the outer harness already keeps synchronized with the scheduler's own clock (Equation (4)). Since the pool is the only piece of state that evolves with simulated time on the path the wait needs, advancing the pool's clock *is* the bounded wait — the controller's wait function is connected directly to the pool's own time-advance call, and the harness extends its scheduler advancement by the same duration to keep the two clocks in lockstep. This also means waiting through an active outage correctly yields no replenishment with no special-casing, since the pool's time-advance call is already defined to be a no-op during an outage. We verified the fix with ten new regression tests, including one specifically constructed so that identical inputs differing only in the wait timeout parameter reach different mode decisions — which was not true before the fix, regardless of the configured timeout. Every number reported in Section XI was generated after this fix; a comparable pilot run before it would have shown near-zero key-establishment latency for B5 under degraded availability, because the wait that now dominates that latency (Figure 5) would never have executed. We surface this not to claim credit for a bug fix, but because Section XIII's broader point — that a results section is only as trustworthy as its instrumentation — is best made concrete rather than asserted in the abstract.

XIII. Threats to Validity and Limitations

We organize this section using the standard internal/external/construct validity taxonomy from empirical software engineering, so that each threat's category — and therefore what kind of future work would address it — is unambiguous, before giving the detailed discussion of each specific limitation.

A. Validity Taxonomy Summary

Internal validity (whether the observed effects are actually caused by what we claim causes them, within this study): the instrumentation defect and fix described in Section XII is the paper's most direct internal-validity concern — it is specifically addressed by the fix and its regression tests, but its prior existence is a reason for appropriate caution about any single-pass empirical result that has not been independently

TABLE XVIII
GO/NO-GO CRITERIA FOR FULL-STUDY EXPANSION, EVALUATED AGAINST PILOT RESULTS.

#	Criterion	Status	Evidence
1	All 30 configurations complete without unhandled errors	Met	Pilot exit code 0; "30 cells, 150 total runs" logged
2	Fairness audit: message counts/overhead differ only where design specifies	Met	Table X-derived overhead matches hand-computed values exactly (Section XI)
3	B5 demonstrably switches modes at 50% availability, not defaulting to one mode	Met	Fallback frequency 3.2–4.7% at $a = 0.5$ (neither 0% nor 100%)
4	Consistency: B5 at $a = 1.0$ tracks B4; B5 at $a = 0.0$ tracks B2	Met, with a caveat	*Mode* tracks correctly (B5 resolves to hybrid at $a = 1.0$, to PQC-only 99.85% of the time at $a = 0.0$); *latency* does not track B2 at $a = 0.0$ (47.9 ms vs. 0.24 ms), because B5's latency includes the bounded wait B2 never performs — this is expected given Section XII's finding, not a fairness violation
5	Observed variance low enough to distinguish conditions from noise at this repetition count	Met	Non-overlapping bootstrap 95% CIs between availability levels (Table XXI)
6	Per-cell runtime used to extrapolate full-matrix feasibility	Met	Table XIV; full 150-run pilot completed in minutes

TABLE XIX
HYPOTHESIS EVALUATION SUMMARY.

#	Prediction	Status
H1	Adaptive mechanism degrades gracefully; static hybrid shows a sharper drop at the same availability levels	**Supported**, and understated: B4 does not merely show a sharper drop, it collapses to near-total failure (0.20%) while B5 sustains 99.68%
H2	Adaptive-switching cost stays within bounds for routine access, may exceed bounds for emergency access specifically	**Untestable as stated**: no explicit emergency-access latency budget was defined in this pilot; latency itself is characterized (Section XI) but not evaluated against a budget
H3	Fallback frequency scales with device count, consistent in pattern with prior power-grid findings, differing in numeric thresholds	**Directionally supported**; numeric threshold comparison against [4] not completed at sufficient source confidence

re-verified after correction. The absence of a formal between-baseline hypothesis test (Section XIII) is a second internal-validity gap: our qualitative conclusions rest on effect sizes we judge too large to be sampling artifacts, not on a confirmed statistical test.

External validity (whether these results generalize beyond this specific study): this is where this paper's limitations are most serious. Every timing was measured on general-purpose x86-64 hardware, not any embedded or IoMT device (Section XIII), so generalization to constrained hardware is explicitly not supported. The network and QKD models are simplified abstractions, not calibrated against a specific deployed system, so generalization to a specific real 6G-edge or QKD deployment's exact numbers is not supported either — only the qualitative behavioral pattern (adaptive fallback preserves availability that static hybrid does not) is claimed to generalize, and even that claim rests on one pilot, not a full study.

Construct validity (whether we are measuring what we intend to measure): the successful-transmission-rate metric conflates key-establishment failure with network-layer packet loss in a single denominator; Section XI reports this explicitly for the "healthy" baselines' residual $\sim$0.3% shortfall, but a reader should be aware the metric does not, on its own, distinguish the two failure causes without consulting the raw per-transaction failure-reason field. Communication overhead as a single scalar per transaction (Section XIII) is a similar construct simplification — it measures total key-establishment cost, not which specific primitive contributed how much of it.

B. This Is a Simulation Study, Not a Hardware Study

Every timing reported in this paper was measured by executing real ML-KEM-768 and ML-DSA-65 operations on the simulation host — a general-purpose x86-64 server (Table X) — not on any embedded or IoMT device. The simulation contains no per-device processing profile, clock-rate parameter, or cycle-count model; every simulated endpoint is, computationally, the host CPU. This is an internally fair basis for the comparative claims this paper makes (every baseline is measured under identical conditions), but it is explicitly *not* a basis for any claim about whether a constrained IoMT device could perform these operations within a real-time deadline, and we make no such claim. Where this paper's architecture (Section V) places signature generation at the Edge Gateway rather than the endpoint, that is a design decision inherited from the broader system model, not an empirical finding this simulation established or tested.

C. Network and QKD Model Simplifications

The three-hop network model (Section IX-E) uses one identical parameter set across all three hops in the reported pilot, which flattens what should realistically be a meaningfully different access-versus-backbone latency and loss profile; this is a known simplification we did not correct before running the pilot reported here. The model implements no 6G protocol stack — no physical or medium-access layer — and 6G standards themselves remain unfinalized, so network parameters throughout this paper are configurable, plausible modeled assumptions, not measurements of any deployed or standardized system. The QKD pool model (Section IX-D) is similarly a modeled resource, not a quantum-physical simulation; its capacity, generation rate, and availability-to-rate mapping are explicit sensitivity variables. Throughput and CPU/memory consumption were not collected as metrics in this pilot. Communication overhead is recorded as a single scalar per transaction; it is not decomposed into per-primitive shares (e.g., how much of B5's 8,890 bytes is attributable to the mode-sync signature specifically versus the session-establishment signature), which would require a small, currently unimplemented instrumentation change.

D. Relationship to URLLC-Class Latency Targets

6G research programs generally carry forward, and in places tighten, the Ultra-Reliable Low-Latency Communication (URLLC) service class defined for 5G, which in its most demanding form targets end-to-end latency on the order of 1 ms at a 99.999% reliability bound for the most latency-critical traffic classes. We deliberately do not adopt a URLLC-class figure as this paper's own pass/fail latency threshold, for two reasons stated here rather than left for a reader to have to infer. First, URLLC's headline numbers describe the radio-access-network hop specifically, under a fully specified 6G physical and medium-access layer that does not exist yet in any finalized standard and that this paper's network model does not attempt to represent (Section IX-E models three generic hops with configurable, not standards-derived, parameters). Applying a URLLC figure to this paper's own end-to-end latency numbers — which include key-establishment, payload encryption, and simulated multi-hop network transit together, not the radio hop alone — would compare a number defined for one specific segment of a real standardized system against a number defined for a different, broader, and non-standardized abstraction; the comparison would be evocative but not meaningful. Second, and more directly relevant to this paper's own results: B5's key-establishment latency alone, at 47.9 ms under $a = 0.0$ (Section XI), already exceeds a 1 ms URLLC-class budget by more than an order of magnitude before any network-transit or payload-encryption cost is added, which is worth stating plainly rather than allowing an unstated implicit comparison to suggest otherwise. This is not evidence the architecture is unfit for a 6G deployment context — URLLC's most demanding figures apply to a narrow slice of traffic (e.g., safety-critical industrial control), and this paper explicitly does not claim EHR emergency access requires that specific bound, only that it requires *some* bound below which its emergency-flagged criticality class is defined, which Section XII's incomplete test of H2 already identifies as undefined in this pilot's own design. Defining an EHR-appropriate latency budget — which may or may not resemble URLLC's own figures — and testing this architecture against it directly is listed as future work (Section XV) rather than answered here by borrowing a number from an adjacent, differently scoped standard.

E. Between-Baseline Hypothesis Testing

No formal statistical test (e.g., Mann-Whitney U, which this project's own methodology had originally specified as the intended approach) has been applied to any comparison in Section XI or Section XII. Every reported difference is descriptive, supported by a bootstrap confidence interval at the individual-cell level but not by a formal between-cell significance test. Given the sample sizes involved (up to 50,000 pooled transactions per cell) and the size of the observed effects (a rate difference of roughly 99.5 percentage points between B4 and B5 at $a = 0.0$), we consider it very unlikely that a formal test would reverse this paper's qualitative conclusions, but we have not performed the test that would confirm this, and we do not claim statistical significance in the formal sense anywhere in this paper.

F. Pilot Scope, Not Full Study

This is a 30-configuration pilot, not the full parameter sweep the broader research design (Section X) specifies. Payload class and network load were held constant (medium, nominal) throughout; a full study would vary both. Adaptive-controller thresholds were fixed at one provisional set of values, not swept or independently validated as well-chosen. The pilot's own stated purpose — producing a variance estimate to inform a full study's repetition count and parameter ranges — has not itself been separately analyzed as a distinct output of this work.

G. Threshold Sensitivity: A Qualitative Argument, Not a Measured Result

The adaptive thresholds used throughout this pilot ($\tau_{\text{hybrid}} = 0.3$, $\tau_{\text{wait}} = 0.1$, $T_{\text{wait}} = 0.05\,\text{s}$) were fixed at one provisional set of values (Table XIII), not swept, so this paper cannot report how B5's results would change under a different setting — that sweep is future work (Section XV). We can, however, reason qualitatively about the direction of that dependence from the mechanism itself (Algorithm 1), without claiming this reasoning as a measured finding. Raising τ_{hybrid} would make the controller more conservative about accepting hybrid mode, pushing more transactions into the wait-or-fallback branch even at availability levels this pilot currently resolves to immediate HYBRID without incident — this would likely raise B5's fallback frequency and mean latency at every availability level below the new, higher threshold, while making the QKD contribution's information-theoretic guarantee available less often. Lowering τ_{hybrid} trades in the opposite direction: fewer transactions wait or fall back, but hybrid mode would be selected under thinner pool margins, closer to the point where debiting the pool for one session could push the next

transaction's request below what it needs. Widening T_{wait} would give more transactions in the wait branch a chance to resolve to HYBRID rather than falling back, at the direct cost of raising the ceiling this paper's latency-distribution finding (Section XI) shows transactions clustering against — the bimodal median-near-ceiling pattern observed at 50 ms would be expected to reappear at whatever new ceiling replaces it, not disappear, since it is a structural property of a bounded-wait-then-fallback design, not an artifact specific to the 50 ms value chosen for this pilot. None of these directional claims should be read as validated; they are the specific, checkable predictions a future threshold-sensitivity sweep (Section XV) would be designed to confirm or refute, stated here in advance for the same pre-registration discipline Section XV's statistical plan already applies to other aspects of this work.

H. Additional Implementation Findings

Three further findings from this project's validation process are recorded here because each carries a real, if currently contained, risk to future work built on this codebase.

First, the recorded key-source label for each transaction (which mode — classical, PQC-only, QKD-only, static hybrid, or adaptive hybrid — actually produced the session key, as distinct from which mode was configured) is serialized via Python's default string conversion of an enumeration member. The current pipeline is internally self-consistent: the aggregator matches the exact string this serialization produces, and the test suite uses the same form. This consistency is, however, an accident of a specific Python enumeration implementation detail rather than a deliberately pinned contract, and a routine-looking refactor of the enumeration type could silently change the string produced, which would cause the fallback-frequency metric (Section XI) to compute as zero rather than raise an error — a silent correctness failure of exactly the kind Section XII's instrumentation-fix narrative warns against. This is flagged, not yet fixed, and should be pinned deliberately (either by matching on the enumeration's explicit value rather than its default string form, or by a regression test asserting the exact serialized string) before further results depend on it.

Second, one code path retains a numeric constant from before the pool-capacity recalibration described in Section IX-D, superseded and explicitly commented as such, but not removed. It is not read by any live code path and does not affect any result in this paper; it is noted here only so a future reader does not mistake it for a live parameter that contradicts the pool sizing this paper actually used.

Third, this project's raw experimental output is committed to the same version-control history as its source code, with no policy yet decided for whether that remains appropriate at full-study scale (Section XV). The pilot's raw output, compressed, is on the order of tens of megabytes; a full study sweeping a substantially larger parameter matrix would produce proportionally more, and a storage-policy decision — version control, a dedicated large-file storage mechanism, or external archival with only aggregated summaries retained in the primary repository — should be made deliberately before that scale is reached, rather than by default.

I. Operational Data-Integrity Note

The metrics collector's output file was originally opened in append mode, so that a single run's many transaction records accumulate correctly without holding the entire run in memory; this project's validation phase identified that this design silently duplicates a configuration's transaction records if the same output location is re-run into without being cleared first, with no error raised. This has since been corrected: the collector now opens its output file in truncate mode, so re-running the same configuration into the same path overwrites rather than duplicates, closing the duplication risk. This trades one hazard for a narrower one — a truncate-on-open collector will silently discard a prior run's data if invoked against the same output path before that data has been copied elsewhere, whereas an append-mode collector would have preserved it (duplicated, but preserved). We consider this the correct trade for this project's needs, since duplication is a silent correctness failure of the kind Section XII treats as unacceptable, while accidental overwrite is a data-management hazard addressable by output-path discipline (e.g., a timestamped or run-identifier-scoped path per invocation) rather than a numeric-correctness one. Every one of the pilot's 150 output files reported in this paper was independently verified to contain zero duplicate transaction identifiers before the results in Section XI were computed, under the corrected collector; any future re-run should still verify output-path uniqueness across runs as a matter of routine, since the truncate-mode fix does not itself guarantee a caller invokes the collector with a distinct path per run.

J. Literature Coverage

As stated in Section III, this paper's literature review rests on approximately 25 targeted searches, not a full systematic review, and several of its most load-bearing related-work citations are held at a partial-verification confidence tier rather than independently full-text confirmed. We consider the research gap identified in Section III well-supported at that confidence level, but a more exhaustive systematic search could in principle surface a directly overlapping prior study that this search did not find.

XIV. Security Considerations

This section is an informal engineering discussion, not a formal security proof, consistent with the explicit non-claim stated in Section VII-D. We discuss each in-scope threat from Section IV in turn.

T1 (harvest-now, decrypt-later). Every mode except the deliberately non-quantum-resistant classical baseline (B1) includes a PQC component (Table IX), so an adversary recording ciphertext for later decryption under a future CRQC gains nothing from B2–B5 traffic that PQC's hardness assumption does not already account for. B1 is not a mitigation against T1 and is not presented as one; it exists solely as a pre-quantum comparison point.

T2 (passive interception). Addressed by AEAD encryption of the application payload (identical AES-256-GCM across every baseline, Section VIII) and, where present, by QKD's

TABLE XX
THREAT-TO-MITIGATION MAPPING.

Threat	Primary architectural mitigation
T1	PQC component in every mode except B1 (Table IX)
T2	AES-256-GCM AEAD, identical across every baseline
T3	ML-DSA authentication of the QKD classical control channel, independent of the quantum channel's own security
T4	Endpoint credential authentication at the Edge Gateway (partial — no content-level anomaly detection)
T5	Adaptive controller (Algorithm 1) routes to PQC-only rather than failing
T6	Fallback target is PQC (quantum-resistant), not an unauthenticated or unencrypted mode

own physical detection property for the quantum channel specifically — a property this paper does not independently verify, since no physical QKD channel exists in this simulation (Section XIII).

T3 (classical-channel MITM against QKD). Addressed by authenticating the QKD classical control channel with ML-DSA independently of the quantum channel's own security, following the design assumption grounded in [8] (Section III). This is, by our own account (Section VII-F), the single most load-bearing design assumption inherited rather than independently re-derived in this paper, and its correctness depends on that source's finding holding up under full verification.

T4 (compromised IoMT endpoint). Partially addressed: the Edge Gateway authenticates the endpoint's credential (Section V), but a device that is compromised while retaining a valid credential can still submit falsified telemetry that authenticates correctly — credential validity is not data-integrity validity, and this paper's trust-boundary model (Section V) treats the endpoint as semi-trusted specifically because of this gap. No content-level anomaly detection is modeled.

T5 (QKD unavailability) and T6 (induced/spoofed unavailability). These are the threats this paper's central mechanism exists to address, and Section XI reports the empirical outcome directly: under QKD unavailability, the adaptive baseline (B5) sustains a 99.68% successful-transmission rate by routing to PQC-only, while the static hybrid baseline (B4) collapses to 0.20%. Because the fallback target is PQC — itself quantum-resistant under the PQC hardness assumption — rather than an unauthenticated, unencrypted, or classical-only mode, an adversary that successfully induces or spoofs QKD unavailability (T6) to force this fallback does not thereby force the system into a broken cryptographic state; they force it into B2's security posture, which is a deliberate, evaluated fallback target, not an accidental weakness.

The hybrid combiner's own security property, as stated in Section VII-D, is not independently proven in this work. Physical-layer QKD attacks, compromise of the PQC hardness assumption itself, and insider threats using legitimate credentials remain explicitly outside this paper's scope (Section IV). Table XX summarizes the mapping between each in-scope threat and the architectural element that addresses it.

A. Residual Risk Summary

No mitigation in Table XX eliminates its corresponding threat outright; each reduces it to a smaller, more specific residual risk, which we state explicitly rather than leave implicit. For T1, the residual risk is that ML-KEM's Module-LWE hardness assumption itself is eventually broken — by a cryptanalytic advance short of a full CRQC, or by a CRQC arriving with capabilities beyond what current parameter selection anticipates — a risk inherited from the broader PQC research program, not one this paper's architecture can independently reduce further. For T2, the residual risk is metadata and traffic-analysis leakage: AEAD protects payload confidentiality and integrity, but transaction timing, size class, and frequency remain observable to a passive eavesdropper positioned on the network path, and this paper's evaluation does not measure or mitigate that channel. For T3, the residual risk is entirely concentrated in one dependency: if the specific claim in [8] that ML-DSA-based classical-channel authentication is sound in this configuration does not hold up under independent verification (a source we hold at partial-verification confidence, Section III), this paper's QKD security guarantee is undermined at its foundation, not merely weakened — this is the single highest-leverage unverified claim in the entire paper, more consequential than any pilot-scale empirical finding, precisely because every other result in this paper implicitly assumes it holds. For T4, the residual risk is the full space of falsified-but-credentialed telemetry, since no content-level anomaly detection is modeled; a compromised device can misreport clinical data indefinitely without triggering any mechanism this architecture provides. For T5 and T6, the residual risk is availability under sustained, not merely transient, QKD denial: the adaptive mechanism converts a QKD outage from a correctness failure into a security-posture degradation (falling back to PQC-only), but it does not restore QKD's information-theoretic guarantee during that outage, and a deployment relying on that guarantee specifically during an adversary-induced outage window receives none of it — the mechanism trades QKD's specific security property for availability precisely when an adversary has forced that trade, which is a deliberate, disclosed design choice (Section VI-E), not an oversight, but it is a real residual risk that should not be understated in any deployment-facing summary of this work.

XV. FUTURE WORK

A. Pre-Registered Statistical Plan for a Full Study

This project's methodology fixed a statistical plan for a full study before the pilot was run, which we report here both for transparency and because the pilot's own results (Section XI) retrospectively confirm one of its central premises. The plan specifies: (i) a repetition count chosen *after* the pilot, via a standard sample-size-for-target-confidence-interval-width calculation applied to the pilot's own observed variance in the primary latency metric — not a habitually chosen round number — which this paper's pilot data (Table XXI) now makes possible to compute, though we have not yet performed that calculation here; (ii) bootstrap rather than normal-approximation confidence intervals throughout, justified in ad-

vance by the expectation that the adaptive controller's bounded wait would produce a right-skewed or bimodal latency distribution (a fast path versus a wait-then-fallback path) for which a normal-distribution interval would misstate the true interval; (iii) both mean and median reported, mean for comparability with conventional overhead/throughput reporting and median for robustness to the anticipated skew; (iv) a default to the non-parametric Mann-Whitney U test for baseline-to-baseline comparisons, with an explicit, data-driven escape hatch to a t-test if the pilot's actual distributions turn out reasonably normal on inspection — a check this paper has not yet performed, and Section XIII accordingly does not claim either test's assumptions are known to hold; and (v) an explicit requirement to apply a multiple-comparisons correction (Bonferroni or a less conservative false-discovery-rate method) once the full study's actual number of pairwise comparisons is known, to avoid overstating findings from chance alone. On point (ii) specifically: B5's pooled distribution at $a = 0.0$, 1,000 devices shows a mean (47.88 ms) below its median (50.38 ms), consistent with a mixture of a fast no-wait path and a slower wait-dominated path rather than a symmetric distribution — the pre-registered expectation that motivated using bootstrap intervals in the first place, now visible in real pilot data rather than only anticipated.

B. Generalizability Beyond EHR Workloads

Every design and evaluation choice in this paper is stated in EHR-specific terms, but the underlying mechanism — an availability-aware controller that monitors a bounded, rate-limited security resource and falls back to a weaker-but-available mode within a criticality-aware bounded wait — is not intrinsically tied to healthcare data. The closest evaluated precedent, Zhu's power-grid analysis [4] (Section III), already applies a structurally similar pattern to GOOSE/PMU-class grid-protection traffic, which shares EHR's combination of a hard latency-sensitive message class (grid protection trip commands; this paper's emergency-flagged transactions) and a latency-tolerant bulk class (grid telemetry logging; this paper's routine transactions) without sharing EHR's regulatory or clinical context at all. This suggests the mechanism's applicability is bounded less by domain (healthcare specifically) than by workload shape: any application with (i) a security requirement strong enough to motivate QKD's information-theoretic guarantee in the first place, (ii) a bounded, rate-limited QKD resource shared across concurrent sessions, and (iii) a meaningful distinction between at least two request-criticality classes with different tolerance for waiting is a structural candidate for this same adaptive pattern, independent of whether the data in transit is a patient record, a grid-protection command, or a financial transaction. We state this as a plausible generalization, not a validated one — this paper evaluated exactly one workload shape (Section IX-F), and confirming the pattern transfers would require a separate evaluation against that other domain's own real traffic characteristics and its own criticality semantics, which are unlikely to be identical to EHR's routine-versus-emergency split even where the general shape is structurally similar. We flag it here specifically so a reader working in an adjacent latency-and-availability-critical domain can judge for themselves whether this architecture's core mechanism, if not its specific EHR-tuned parameters (Table XIII), is worth adapting rather than reinventing.

Directly motivated by the limitations in Section XIII: (1) extend the network model so the three simulated hops carry independently configured parameters, giving a realistic access-versus-backbone distinction; (2) collect throughput and CPU/memory metrics, and decompose communication overhead by contributing primitive; (3) apply a formal between-baseline hypothesis test (Mann-Whitney U, as originally specified) to the pooled pilot data or a subsequent full study; (4) run the full parameter sweep this pilot was designed to calibrate — varying payload class, network load, and a broader QKD availability range, and sweeping the adaptive-controller thresholds themselves as an independent variable rather than holding them fixed; (5) define an explicit, justified emergency-access latency budget and re-evaluate H2 against it directly, which this pilot's design did not include; (6) obtain independently verified numeric thresholds from [4] (or a comparable power-grid-domain source) sufficient to complete the direct numeric comparison originally specified as Contribution C3; (7) map the architecture's QKD subsystem interface onto the relevant ETSI QKD API standardization work, which the current implementation does not attempt; (8) extend the literature search toward the fuller systematic review this paper's related-work section falls short of; (9) evaluate device-class-specific timing — explicitly not attempted here (Section XIII) — as a separate, hardware-grounded study if the architecture's endpoint-versus-gateway signature placement decision is to be empirically justified rather than inherited by design.

XVI. Conclusion

We presented an adaptive QKD-PQC key-establishment architecture for EHR sharing in a 6G-edge healthcare topology, implemented it alongside four baselines using real ML-KEM-768 and ML-DSA-65 operations in a validated discrete-event simulation, and evaluated all five in a 30-configuration, 757,500-transaction pilot.

A. Practical Implications

For a system architect evaluating whether to adopt an availability-aware adaptive fallback over a simpler static hybrid design, this paper's results give a concrete, if provisional, answer to what that choice costs and buys: adopting the adaptive mechanism buys a successful-transmission rate that holds near 99.7% through QKD degradation and outage where the static alternative collapses to near-total failure, at a fixed key-establishment communication-overhead cost of one additional signature round trip (Table IX) and a latency cost that is negligible when QKD is healthy but can reach tens of milliseconds — driven by a configurable bounded-wait policy, not by cryptography — when it is not. Whether that latency cost is acceptable depends entirely on the deployment's own latency budget for the affected transaction class, which this pilot's design did not itself define (Section XII's incomplete

test of H2) and which we therefore leave as a deployment-specific judgment this paper informs but does not make.

B. Returning to the Motivating Scenario

Section I opened with a hospital edge gateway serving both routine and emergency EHR access over a QKD link shared with the hospital's classical infrastructure, and asked what a static hybrid design would cost that link's users if the QKD channel degraded. The pilot's results answer that question concretely, not just qualitatively: under the scenario's worst case — QKD unavailable entirely — a static hybrid gateway (B4) would fail 99.80% of the sessions attempted through it, including, indiscriminately, the emergency department's request from that scenario, since B4's design draws no distinction between transaction criticality classes at all. The adaptive gateway (B5) would instead complete 99.68% of those same sessions, at a measured key-establishment latency cost of approximately 48 ms for the routine majority that pay the bounded wait, and with the emergency-flagged transaction specifically routed around that wait entirely by Algorithm 1's criticality branch (Section VI-B) — exactly the differentiated treatment the scenario's emergency-department vignette motivated. Whether 48 ms of additional latency is acceptable for the routine majority is, as stated in the following subsection, a deployment-specific judgment this paper informs but does not make; what this paper does establish is that the scenario's central tension — resilience against QKD degradation versus the cost of achieving it — is not hypothetical, is quantifiable from a real, tested implementation rather than only argued from first principles, and resolves clearly in the adaptive design's favor on the one metric (successful transmission during degradation) the scenario's emergency case cared about most.

C. Concluding Remarks

The central result is a sharp, structurally enforced behavioral divergence: a static QKD-PQC hybrid baseline collapses to a 0.20% successful-transmission rate under QKD unavailability, indistinguishable from an unmitigated QKD-only baseline, while the adaptive mechanism sustains 99.68% by falling back to a PQC-only path — at a measured cost of roughly two orders of magnitude in key-establishment latency under degraded availability, driven by the controller's bounded wait rather than by cryptographic cost, and a fixed key-establishment overhead of 8,890 bytes regardless of which mode is ultimately selected. We reported this result together with the instrumentation defect that, until corrected and verified, would have prevented it from being measured at all — because a results section is only as trustworthy as the pipeline that produced it, and we consider that account part of this paper's contribution, not an incidental footnote to it. This is a simulation study with real cryptography on general-purpose hardware; it makes no claim about embedded device performance, and Section XIII states every other boundary of what it does and does not establish. Within that stated scope, the evidence supports the paper's central architectural claim: an availability-aware adaptive fallback between QKD-PQC hybrid and PQC-only key establishment is both necessary and effective for EHR sharing under realistic QKD channel degradation, where a static hybrid alternative is not.

Appendix A
Full Pilot Results, All 30 Configurations

Table XXI reports the complete pooled pilot summary: all 5 baselines × 3 QKD availability levels × 2 device counts, each pooled across 5 repetitions. This extends Table XVI (which reported only the 1,000-device subset) to the full matrix, including the 10-device cells.

TABLE XXI
COMPLETE POOLED PILOT RESULTS, ALL 30 CONFIGURATIONS.

BL	a	Devices	Success/n	key-establishment (ms)	E2E (ms)	Fallback
B1	0.0	10	498/500	0.3040	11.763	—
B1	0.5	10	500/500	0.2896	11.746	—
B1	1.0	10	499/500	0.3104	11.750	—
B1	0.0	1000	49848/50000	0.2961	11.740	—
B1	0.5	1000	49856/50000	0.2985	11.737	—
B1	1.0	1000	49858/50000	0.2932	11.736	—
B2	0.0	10	499/500	0.2344	11.568	—
B2	0.5	10	499/500	0.2336	11.627	—
B2	1.0	10	500/500	0.2412	11.661	—
B2	0.0	1000	49839/50000	0.2422	11.689	—
B2	0.5	1000	49849/50000	0.2468	11.689	—
B2	1.0	1000	49850/50000	0.2429	11.687	—
B3	0.0	10	100/500	0.1614	11.601	—
B3	0.5	10	345/500	0.1762	11.689	—
B3	1.0	10	497/500	0.1630	11.587	—
B3	0.0	1000	100/50000	0.1852	11.687	—
B3	0.5	1000	25018/50000	0.1733	11.614	—
B3	1.0	1000	49839/50000	0.1665	11.608	—
B4	0.0	10	100/500	0.2752	11.749	—
B4	0.5	10	344/500	0.2634	11.688	—
B4	1.0	10	499/500	0.2556	11.681	—
B4	0.0	1000	100/50000	0.2618	11.581	—
B4	0.5	1000	25033/50000	0.2887	11.732	—
B4	1.0	1000	49850/50000	0.2626	11.703	—
B5	0.0	10	497/500	40.2358	51.646	0.8500
B5	0.5	10	499/500	32.8894	44.245	0.0320
B5	1.0	10	496/500	0.4380	11.835	0.0000
B5	0.0	1000	49842/50000	47.8768	59.324	0.9985
B5	0.5	1000	49862/50000	45.5832	57.024	0.0470
B5	1.0	1000	49865/50000	0.4323	11.877	0.0000

Appendix B
Reproducibility

Every value in Table XXI is reproducible from a single top-level random seed. The environment (Table X) and the exact pilot configuration — three QKD availability levels, two device counts, one payload class, one network load, five baselines, five repetitions per cell, adaptive thresholds $\tau_{\text{hybrid}} = 0.3$, $\tau_{\text{wait}} = 0.1$, $T_{\text{wait}} = 0.05\,\text{s}$, and base seed 42 — are recorded in full in the project's version-controlled configuration file. Determinism was independently verified by executing an identical cell twice under identical seed, configuration, and workload: the two runs produced zero mismatches across every deterministic field (transaction identifiers, payload sizes, selected mode, controller state, success outcome, and simulated timestamp) despite genuinely exercising the bounded-wait path in both runs. The pilot is executed via a single command against the recorded configuration and produces raw, per-transaction JSON-lines output before any aggregation, per the metrics discipline described in Section IX.

Table XXII summarizes, for each standard reproducibility dimension, what this project provides.

TABLE XXII
REPRODUCIBILITY CHECKLIST.

Dimension	Provided
Deterministic seeding	One top-level seed; independent per-component sub-seeds derived from it
Environment specification	Exact resolved dependency versions and platform (Table X)
Cryptographic library provenance	liboqs source commit identifier, build flags, minimal algorithm selection
Configuration provenance	Full pilot configuration version-controlled and referenced by this appendix
Raw data before aggregation	Per-transaction JSON-lines records; all descriptive statistics computed from these, never the reverse
Determinism verification	Independently re-executed; zero mismatches across all deterministic fields
Data-integrity check	All 150 pilot output files independently verified free of duplicate transaction identifiers before use
Known instrumentation history	The pre-fix defect and its fix are documented (Section XII), not silently omitted

APPENDIX C
SOFTWARE ARCHITECTURE

The simulation is organized into eight modules, summarized in Table XXIII, structured specifically so that the cryptographic, network, workload, adaptive-decision, and metrics concerns can each be inspected, tested, and modified independently.

TABLE XXIII
SOFTWARE MODULE STRUCTURE.

Module	Contents
Cryptography	Classical (X25519/Ed25519), PQC (ML-KEM/ML-DSA via liboqs), QKD pool, hybrid HKDF combiner, authentication
Network	Topology, per-hop channel model
Workload	Synthetic EHR transaction generator
Adaptive	The controller implementing Algorithm 1
Baselines	B1–B5, sharing one interface and one AEAD implementation
Simulation	Discrete-event harness, per-transaction event records
Metrics	Raw-record collector; per-cell aggregator (percentiles, bootstrap CI)
Utilities	Configuration loading, seed management, logging

APPENDIX D
PILOT CONFIGURATION FILE

For completeness, and because a configuration file left only as prose description invites transcription drift between what a paper says a study used and what its code actually ran, we reproduce the pilot's version-controlled configuration file verbatim below (`config/pilot.yaml`), rather than only paraphrasing its contents in Section X. Every numeric value discussed in this paper's methodology, results, and worked-example (Section VI-B) sections traces to this single file.

```
experiment_name: pilot

qkd_availability_levels: [1.0, 0.5, 0.0]
device_counts: [10, 1000]
payload_classes: [medium]
network_loads: [nominal]
baselines: [B1, B2, B3, B4, B5]

n_transactions_per_device: 10
sim_duration_seconds: 10.0

repetitions_per_cell: 5

adaptive_thresholds:
  pool_min_hybrid: 0.3
  pool_min_wait: 0.1
  wait_timeout_seconds: 0.05

qkd_pool_sessions_buffer: 20

base_seed: 42

output_dir: results/raw/pilot
```

Two fields deserve a note beyond what the file's own inline comments (omitted above for brevity) state. First, `qkd_pool_sessions_buffer: 20` combined with the fixed 256-bit per-session QKD draw (Table XIII) yields the pool capacity $C = 20 \times 256 = 5{,}120$ bits used throughout Section IX-D; the configuration file expresses capacity indirectly, as a buffer-of-sessions quantity, rather than as a raw bit count, specifically so that resizing the pool in terms meaningful to an operator ("how many sessions of headroom") does not require recomputing a bit count by hand. Second, `base_seed: 42` is the single value from which every other random draw in the pilot — per-repetition seeds, per-transaction workload sampling, network-loss sampling — is derived deterministically via the seed-management utility described in Table XXIII's Utilities row; changing this one value reproduces a structurally identical but numerically distinct pilot, which is the mechanism Appendix B's determinism verification relies on.

REFERENCES

[1] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," FIPS 203, 2024.

[2] ——, "Module-lattice-based digital signature standard," FIPS 204, 2024.

[3] M. Clason, J. Argillander, D. Bergstrom, D. Spegel-Lexne *et al.*, "Deployed trusted-node quantum key distribution over 300 km with a multi-site network," arXiv:2606.06107, 2026.

[4] Y. Zhu, "Techno-economic feasibility analysis of QKD for power-system communications," arXiv:2510.15248, 2025.

[5] P. Devaraj, S. A. Chaman Basha, N. N. Panarkuzhiyil Santhosh, and N. Panda, "A post-quantum secure architecture for 6G-enabled smart hospitals: A multi-layered cryptographic framework," *Future Internet*, vol. 18, no. 3, p. 165, 2026.

[6] D. Roosan *et al.*, "Post-quantum cryptography resilience in telehealth using quantum key distribution," *Blockchain in Healthcare Today*, 2025.

[7] P. Papadopoulos, T. Korakis, and N. Balaskas, "Practical deployment of hybrid QKD and PQC for off-site hospital data," *IEEE Access*, 2026, early Access.
[8] A. Atutxa, M. Sanz *et al.*, "Authentication of the QKD classical channel through PQC in a multi-site 5G/6G quantum-safe network," venue not independently confirmed, 2025.
[9] "Post-quantum cryptography for healthcare: securing medical data, connected devices, and digital health infrastructure," *Frontiers in Health Services*, 2026.
[10] "Post-quantum authentication in the internet of medical things: A system-level review and future directions," *Computers*, vol. 15, no. 3, p. 189, 2026.
[11] J. Spooren *et al.*, "PQC-enhanced QKD networks: A layered approach," in *Proc. IEEE Int. Conf. Quantum Communications, Networking, and Computing (QCNC)*, 2026.
[12] S. Mahesh and D. Mishra, "PCBQC: Blockchain-based patient-centric EHR using hybrid PQC lattice algorithms," *International Journal of Performability Engineering*, vol. 21, no. 11, 2025.
[13] F. Maqsood, A. Hameed, M. Junaid, and S. Tahir, "Enhancing IoT healthcare security with post-quantum cryptography," IEEE Xplore document 10937815, 2025.
[14] "Secure medical data transmission using QKD and PQC in real-world fiber networks," arXiv:2608.18869, 2026.
[15] A. Kudaloor and A. Aijaz, "Toward quantum-safe 6G: Experimental evaluation of post-quantum cryptography," arXiv:2605.06881, 2026.
[16] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, IETF, 2010.